Matthew Helbig

Factors Influencing Hotel Reservation Cancellations: A Logistic Regression Approach

Analyzing and Predicting Hotel Reservation Cancellations Using Logistic Regression

Western Governors University

January 8th, 2025

A. Research Question

In the hospitality industry, hotels pay close attention to reservation cancellations, as unaddressed cancellations can lead to an inefficient allocation of hotel rooms and potentially decreased revenue for the hotel. According to Skift.com (2024), "TMCs [travel management companies] have seen an average annual revenue loss of 6.7 percent over the past five years due to unaccounted-for cancellations and no-shows." Hotels and companies that book hotel travel are greatly impacted by the impact of reservation cancellations, therefore it would stand to be beneficial to a hotel to develop the ability to accurately predict the likelihood of any one particular reservation being cancelled.

Furthering the pressing need for an analysis of hotel cancellations is the increase in customers attempting to "game" the system to find optimal deals and room rates. Zvi Schwartz, Ph.D. at Boston University (2021) notes that "A growing number of customers have learned over the years that the strategy of 'book and search' could be beneficial. They book a room, reduce their risk of not having a room, but then continue to search for a better deal. When they find a better deal, they cancel their initial reservation and book the better deal. Note that the new better deal reservation could be with a lower rate at the same hotel, or it could be a room in another hotel that the customer perceives to be of a better value." As customers develop new methods for bargain hunting that rely on cancelling reservations, hotels are behooved to develop revenue-retaining strategies in order to stay competitive. Identifying factors that most impact hotel reservation cancellations can help the hotel to stay ahead of the curve and mitigate any potential revenue loss from cancellations.

This analysis seeks to determine whether factors leading to hotel reservation cancellations can be identified, as well as to determine the likelihood that any one reservation will be canceled given a set of identifying factors. As noted by Skift and Schwartz, this analysis could provide significant revenue protection as well as revenue gains for the hotel, and occurs at a vital time for the hospitality industry as a whole. In short, the analysis will attempt to answer the following question: what factors most significantly influence hotel reservation cancellations, and how accurately can logistic regression predict the likelihood of a reservation being canceled? The null hypothesis for this analysis is that no identified variables have a significant influence on the likelihood that a hotel reservation is canceled. The alternative hypothesis is that at least one

of the identified independent variables has a significant influence on the likelihood of hotel reservation cancellations.

B. Data Collection

In order to facilitate the data analysis, a data set was obtained from Kaggle.com titled "Hotel Reservations Dataset" (Raza, 2023). This dataset contains 36,275 observations across 19 columns. These columns include a mix of categorical and numerical datatypes that will be further analyzed in the data extraction and preparation process. There are fourteen numerical variables and five categorical variables. In addition to the existing variables, new variables will be created as part of the data extraction and preparation process in order to help facilitate a more accurate and robust analysis. The target variable for the logistic regression model is whether or not the hotel reservation was canceled. One advantage of using this data is that it provides a mix of both categorical and continuous numerical data, allowing for a variety of potential options for types of variables that can impact the target outcome variable. According to Littler (2021), "When the response variable is binary or categorical a standard linear regression model can't be used, but we can use logistic regression models instead. These alternative regression models are specifically designed for analysing binary (e.g., yes/no) or categorical (e.g., Full-time/Part-time/Retired/Unemployed) response variables. Similar to linear regression models, logistic regression models can accommodate continuous and/or categorical explanatory variables as well as interaction terms to investigate potential combined effects of the explanatory variables." As the chosen data set has a mix of continuous and categorical explanatory variables, and as there exists the potential in the data to create new binary variables and interaction variables, this data set provides a distinct advantage for use in the desired logistic regression model.

While the chosen data set does provide a specific advantage for the data analysis process, it also provides an unfortunate disadvantage: the lack of a data dictionary. Regarding the value of having a data dictionary, Splunk (2024) states that "A data dictionary provides additional context and information about each data point so that analysts can understand the data better." While many columns, primarily numerical in nature, can be understood for the most part by looking at the data, there are categorical columns in the chosen data set that cannot be intuited so simply. Specifically, the type of meal plan chosen by guests and the room type reserved by guests are variables that could theoretically provide valuable insight, however without a data dictionary to

explain what each of the encoded values mean, it is difficult to properly understand the explicit differences between the potential values of these columns.

While the lack of a data dictionary does present a specific challenge, this challenge can be mitigated in a variety of ways. In this instance, the meal plan chosen can be converted into a binary variable that simply indicates whether or not a meal plan was chosen. While there is some information loss in terms of not being able to compare types of meal plans and the difference in impact each chosen meal plan may have, the potential for valuable insight can still be retained in the form of any potential difference in cancellation rates between guests who selected a meal plan and guests did not. Unfortunately, the same strategy cannot be used for the room type reserved, as all guests reserved a room and a binary variable cannot be derived from this column. While this specific challenge cannot be overcome for this variable, we can attempt to use the remaining variables (as well as any newly created variables) to reach the desired outcome of the data analysis process.

C. Data Extraction and Preparation

Once the data is downloaded from the necessary Kaggle repository, the next step is to prepare the data in such a way that the analysis can be performed. For this particular project, the preparation and analysis will be performed using the Python programming language. Python is particularly useful for logistic regression because of the amount of useful libraries it contains. Regarding this, Stojiljković (2023) notes that "There are several packages you'll need for logistic regression in Python. All of them are free and open-source, with lots of available resources." Those packages will be loaded below and used to perform this analysis.

```
In [1]: #Import Libraries
        import pandas as pd
        import scipy.stats as stats
        import matplotlib.pyplot as plt
        import statsmodels.api as sm
        import seaborn as sns
        from statsmodels.stats.outliers_influence import variance_inflation_factor
        import numpy as np
        from sklearn.metrics import confusion_matrix

C:\Users\matth\anaconda3\lib\site-packages\pandas\core\computation\expressions.py:21: UserWarning: Pandas requires version
'2.8.4' or newer of 'numexpr' (version '2.8.3' currently installed).
  from pandas.core.computation.check import NUMEXPR_INSTALLED
C:\Users\matth\anaconda3\lib\site-packages\pandas\core\arrays\masked.py:60: UserWarning: Pandas requires version '1.3.6' o
r newer of 'bottleneck' (version '1.3.5' currently installed).
  from pandas.core import (
C:\Users\matth\AppData\Local\Temp\ipykernel_12488\3315401272.py:2: DeprecationWarning:
Pyarrow will become a required dependency of pandas in the next major release of pandas (pandas 3.0),
(to allow more performant data types, such as the Arrow string type, and better interoperability with other libraries)
but was not found to be installed on your system.
If this would cause problems for you,
please provide us feedback at https://github.com/pandas-dev/pandas/issues/54466

  import pandas as pd
```

Figure 1: Import Libraries

The first step needed is to import the required libraries to perform the analysis. Each of the libraries noted above will be used for specific aspects of the data analysis.

```
In [2]: #Read in data
        hotel_data = pd.read_csv('Hotel Reservations.csv')
```

Figure 2: Read in Data

After the libraries have been loaded, the pandas library is used to read the initial Kaggle data into our programming environment.

```
In [3]: #Duplicate check
        print(hotel_data.duplicated().value_counts())

        False    36275
        Name: count, dtype: int64
```

Figure 3: Duplicate Value Check

```
In [4]: #Missing value check
        print(hotel_data.isnull().sum())

        Booking_ID                              0
        no_of_adults                            0
        no_of_children                          0
        no_of_weekend_nights                    0
        no_of_week_nights                       0
        type_of_meal_plan                       0
        required_car_parking_space              0
        room_type_reserved                      0
        lead_time                               0
        arrival_year                            0
        arrival_month                           0
        arrival_date                            0
        market_segment_type                     0
        repeated_guest                          0
        no_of_previous_cancellations            0
        no_of_previous_bookings_not_canceled    0
        avg_price_per_room                      0
        no_of_special_requests                  0
        booking_status                          0
        dtype: int64
```

Figure 4: Null Value Check

```
In [5]: print(hotel_data.isna().sum())

Booking_ID                            0
no_of_adults                          0
no_of_children                        0
no_of_weekend_nights                  0
no_of_week_nights                     0
type_of_meal_plan                     0
required_car_parking_space            0
room_type_reserved                    0
lead_time                             0
arrival_year                          0
arrival_month                         0
arrival_date                          0
market_segment_type                   0
repeated_guest                        0
no_of_previous_cancellations          0
no_of_previous_bookings_not_canceled  0
avg_price_per_room                    0
no_of_special_requests                0
booking_status                        0
dtype: int64
```

Figure 5: NA Value Check

Figures 3, 4, and 5 represent different assessments of the data's completion, as well as any immediately obvious issues with repeated data. In Figure 3, we see that there are no duplicate values in the data. Figures 4 and 5 indicate that none of the initial values are null or have a value of NA.

```
In [6]: #Print data types for quantitative versus categorical data check
        print(hotel_data.dtypes)

Booking_ID                            object
no_of_adults                           int64
no_of_children                         int64
no_of_weekend_nights                   int64
no_of_week_nights                      int64
type_of_meal_plan                     object
required_car_parking_space             int64
room_type_reserved                    object
lead_time                              int64
arrival_year                           int64
arrival_month                          int64
arrival_date                           int64
market_segment_type                   object
repeated_guest                         int64
no_of_previous_cancellations           int64
no_of_previous_bookings_not_canceled   int64
avg_price_per_room                   float64
no_of_special_requests                 int64
booking_status                        object
dtype: object
```

Figure 6: Datatype Check

Next, we evaluate the datatypes for the variables in the data frame. Immediately, we can see that the target variable (booking_status) is noted as an object. This variable will need to be modified so that it can be used as the target outcome variable for the logistic regression model.

```
In [7]: #Booking status should be a binary outcome with Canceled as 1 and Not_Canceled as 0
        # Reclassify the 'Status' column
        hotel_data['booking_status'] = hotel_data['booking_status'].map({'Canceled': 1, 'Not_Canceled': 0})
        print(hotel_data.dtypes)

Booking_ID                              object
no_of_adults                             int64
no_of_children                           int64
no_of_weekend_nights                     int64
no_of_week_nights                        int64
type_of_meal_plan                       object
required_car_parking_space               int64
room_type_reserved                      object
lead_time                                int64
arrival_year                             int64
arrival_month                            int64
arrival_date                             int64
market_segment_type                     object
repeated_guest                           int64
no_of_previous_cancellations             int64
no_of_previous_bookings_not_canceled     int64
avg_price_per_room                     float64
no_of_special_requests                   int64
booking_status                           int64
dtype: object
```

Figure 7: Target Variable Data Change

Figure 7 above shows the remapping of the target variable from an object to an integer, with values of 1 if the reservation was canceled and 0 if the reservation is not canceled. As we are attempting to assess whether we can predict the likelihood that a reservation is canceled, it intuitively makes sense to have a canceled reservation correspond to a binary value of 1 and a non-canceled reservation correspond to a binary value of 0.

```
In [8]: hotel_data.head()
Out[8]:
```

	Booking_ID	no_of_adults	no_of_children	no_of_weekend_nights	no_of_week_nights	type_of_meal_plan	required_car_parking_space	room_type_reser
0	INN00001	2	0	1	2	Meal Plan 1	0	Room_Ty
1	INN00002	2	0	2	3	Not Selected	0	Room_Ty
2	INN00003	1	0	2	1	Meal Plan 1	0	Room_Ty
3	INN00004	2	0	0	2	Meal Plan 1	0	Room_Ty
4	INN00005	2	0	1	1	Not Selected	0	Room_Ty

Figure 8: Brief Look at Data

Before the next step in the analysis, it can be beneficial to take a brief look at the layout of the data in order to better understand it. Here, we can see a brief look at the typical format that the data will present itself in.

```
In [9]: #Interested in seeing unique value counts for each quantitative variable,
        #maybe we could recode some as binary if it would be more helpful for interpretation
        print(hotel_data.no_of_adults.value_counts())

no_of_adults
2    26108
1     7695
3     2317
0      139
4       16
Name: count, dtype: int64

In [10]: print(hotel_data.no_of_children.value_counts())

no_of_children
0     33577
1      1618
2      1058
3        19
9         2
10        1
Name: count, dtype: int64

In [11]: print(hotel_data.no_of_weekend_nights.value_counts())

no_of_weekend_nights
0    16872
1     9995
2     9071
3      153
4      129
5       34
6       20
7        1
Name: count, dtype: int64

In [12]: print(hotel_data.no_of_week_nights.value_counts())

no_of_week_nights
2     11444
1      9488
3      7839
4      2990
0      2387
5      1614
6       189
7       113
10       62
8        62
9        34
11       17
15       10
12        9
14        7
13        5
17        3
16        2
Name: count, dtype: int64
```

Figure 9: Value Counts 1

Above, we can see the value counts for the Number of Adults, Number of Children, Number of Weekend Nights, and Number of Week Nights variables. This helps to give us an idea of the spread of the data, as well as assists with identifying any results that may stick out as incorrect.

```
In [13]: print(hotel_data.required_car_parking_space.value_counts())

required_car_parking_space
0    35151
1     1124
Name: count, dtype: int64

In [14]: print(hotel_data.lead_time.value_counts()) #Will need to explain what this is in report

lead_time
0      1297
1      1078
2       643
3       630
4       628
       ...
300       1
353       1
328       1
352       1
351       1
Name: count, Length: 352, dtype: int64

In [15]: print(hotel_data.arrival_year.value_counts()) #Could there be more or less based on year

arrival_year
2018    29761
2017     6514
Name: count, dtype: int64

In [16]: print(hotel_data.arrival_month.value_counts()) #More or less cancellations during summer?

arrival_month
10    5317
9     4611
8     3813
6     3203
12    3021
11    2980
7     2920
4     2736
5     2598
3     2358
2     1704
1     1014
Name: count, dtype: int64
```

Figure 10: Value Counts 2

Figure 10 shows the value counts for whether the reservation required a parking space, the lead time for the reservation (how far out in days the reservation was booked), the year of arrival for the reservation, and the month of arrival for the reservation. Understanding the values that these variables can lead to potential observations between the independent variables and the target variable that may be useful for the practicality of the logistic regression model.

```
In [17]: print(hotel_data.arrival_date.value_counts()) #More or less cancellations during beginning, middle, end of month?

arrival_date
13    1358
17    1345
2     1331
4     1327
19    1327
16    1306
20    1281
15    1273
6     1273
18    1260
14    1242
30    1216
12    1204
8     1198
29    1190
21    1158
5     1154
26    1146
25    1146
1     1133
9     1130
28    1129
7     1110
24    1103
11    1098
3     1098
10    1089
27    1059
22    1023
23     990
31     578
Name: count, dtype: int64

In [18]: print(hotel_data.repeated_guest.value_counts()) #Do repeated guests cancel less?

repeated_guest
0    35345
1      930
Name: count, dtype: int64

In [19]: print(hotel_data.no_of_previous_cancellations.value_counts()) #Could re-map to "Has the guest cancelled before?" and add as

no_of_previous_cancellations
0     35937
1       198
2        46
3        43
11       25
5        11
4        10
13        4
6         1
Name: count, dtype: int64
```

Figure 11: Value Counts 3

Figure 11 shows the value counts for the date of arrival, whether the reservation is for a repeated guest, and the number of previous cancellations for the guest. Above, we can see that an overwhelming majority of guests are not repeat guests. This can potentially be taken into consideration during the development of new variables, as well as during the analysis phase.

```
In [20]: print(hotel_data.no_of_previous_bookings_not_canceled.value_counts())

no_of_previous_bookings_not_canceled
0     35463
1       228
2       112
3        80
4        65
5        60
6        36
7        24
8        23
10       19
9        19
11       15
12       12
14        9
15        8
16        7
13        7
18        6
20        6
21        6
17        6
19        6
22        6
25        3
27        3
24        3
23        3
44        2
29        2
48        2
28        2
30        2
32        2
31        2
26        2
46        1
55        1
45        1
57        1
53        1
54        1
58        1
41        1
40        1
43        1
35        1
50        1
56        1
33        1
37        1
42        1
51        1
38        1
34        1
39        1
52        1
49        1
47        1
36        1
Name: count, dtype: int64
```

Figure 12: Value Counts 4

Figure 12 above shows the value counts for the number of previous bookings that were not canceled. From the distribution of this data, we can see that this data has significant overlap with the Repeated Guest variable. This could indicate multicollinearity, which we will look out for later in the data analysis.

```
In [21]: print(hotel_data.avg_price_per_room.value_counts())

avg_price_per_room
65.00     848
75.00     826
90.00     703
95.00     669
115.00    662
          ...
212.42      1
83.48       1
70.42       1
130.99      1
167.80      1
Name: count, Length: 3930, dtype: int64

In [22]: print(hotel_data.no_of_special_requests.value_counts()) #Could re-map to "Made a special request?" and add as a new co

no_of_special_requests
0    19777
1    11373
2     4364
3      675
4       78
5        8
Name: count, dtype: int64

In [23]: print(hotel_data.booking_status.value_counts())

booking_status
0    24390
1    11885
Name: count, dtype: int64
```

Figure 13: Value Counts 5

In Figure 13 above, we can see that the Average Price Per Room variable may need significant wrangling if it is to be used as a predictive variable, as there are 3,930 unique values for this variable. Additionally, we can see that there are approximately as many reservations with special requests as without, which could mean that this variable would be useful as a binary variable. Finally, we see that a majority of the reservations have a Booking Status of 0, meaning that a large majority of reservations were not canceled.

```
In [24]: #Combine certain columns to see if when combined they have more predictive power
         #'Length of stay', 'Count of guests', 'Count of Previous Bookings'
         hotel_data['length_of_stay'] = hotel_data['no_of_weekend_nights'] + hotel_data['no_of_week_nights']
         hotel_data['count_of_guests'] = hotel_data['no_of_adults'] + hotel_data['no_of_children']
         hotel_data['count_of_previous_bookings'] = hotel_data['no_of_previous_cancellations'] + hotel_data['no_of_previous_boo

In [25]: print(hotel_data.length_of_stay.value_counts())

         length_of_stay
         3     10052
         2      8472
         1      6604
         4      5893
         5      2589
         6      1031
         7       973
         8       179
         9       111
         10      109
         0        78
         11       39
         14       32
         15       31
         12       24
         13       18
         20       11
         19        6
         16        6
         17        5
         21        4
         18        3
         23        2
         22        2
         24        1
         Name: count, dtype: int64

In [26]: print(hotel_data.count_of_guests.value_counts())

         count_of_guests
         2     23942
         1      7552
         3      3851
         4       912
         5        15
         12        1
         10        1
         11        1
         Name: count, dtype: int64
```

Figure 14: Value Counts 6

```
In [27]: print(hotel_data.count_of_previous_bookings.value_counts())

         count_of_previous_bookings
         0     35345
         1       322
         2       113
         3        78
         4        60
               ...
         57        1
         37        1
         54        1
         36        1
         35        1
         Name: count, Length: 63, dtype: int64
```

Figure 15: Value Counts 7

Figures 14 and 15 represent an important aspect of the data preparation process, as multiple variables are combined into new variables in order to have these new variables provide potentially more explanatory power than the initial variables they are derived from. The number of week nights and weekend nights have been combined into length of stay, the number adults

and children on a reservation have been combined into the number of guests, and the number of previous bookings canceled/not canceled have been combined into the count of previous bookings.

It is important to note and remember that as these new variables are derived from existing variables, multicollinearity will occur between the variables. This can and will be addressed during the Analysis portion of the process.

```
In [28]: #Create new columns for 'Has the guest cancelled before?' and 'Made a special request?'
         hotel_data['prior_cancellation'] = hotel_data['no_of_previous_cancellations'].apply(lambda x: 0 if x == 0 else 1)
         print(hotel_data.prior_cancellation.value_counts())

         prior_cancellation
         0    35937
         1      338
         Name: count, dtype: int64

In [29]: hotel_data['special_request_flag'] = hotel_data['no_of_special_requests'].apply(lambda x: 0 if x == 0 else 1)
         print(hotel_data.special_request_flag.value_counts())

         special_request_flag
         0    19777
         1    16498
         Name: count, dtype: int64
```

Figure 16: Binary Variable Creation

In Figure 16 above, binary variables are created in order to potentially provide simpler and more interpretable independent variables that can be used to predict the likelihood of cancelation. Prior Cancellation indicates whether a guest has ever canceled a reservation before, while Special Request Flag indicates whether there is a special request attached to the reservation. The thought behind the creation of these binary variables is that the presence of any prior cancelation or special request is more relevant to cancelation status than the count of the cancelations or special requests themselves.

```
In [30]: #Outlier check for quantitative continuous variables
         #All values seem legitimate so not really a need to treat outliers

         #no_of_adults
         ##Outlier assessment via histogram
         hotel_data['Z_Score_no_of_adults']=stats.zscore(hotel_data['no_of_adults'])
         hotel_data[['no_of_adults', 'Z_Score_no_of_adults']].head

Out[30]: <bound method NDFrame.head of        no_of_adults  Z_Score_no_of_adults
         0                 2             0.298893
         1                 2             0.298893
         2                 1            -1.628975
         3                 2             0.298893
         4                 2             0.298893
         ...             ...                  ...
         36270             3             2.226761
         36271             2             0.298893
         36272             2             0.298893
         36273             2             0.298893
         36274             2             0.298893

         [36275 rows x 2 columns]>

In [31]: plt.hist(hotel_data['Z_Score_no_of_adults'])

Out[31]: (array([1.3900e+02, 0.0000e+00, 7.6950e+03, 0.0000e+00, 0.0000e+00,
                 2.6108e+04, 0.0000e+00, 2.3170e+03, 0.0000e+00, 1.6000e+01]),
          array([-3.55684355, -2.78569631, -2.01454908, -1.24340184, -0.47225461,
                 0.29889263,  1.07003987,  1.8411871 ,  2.61233434,  3.38348157,
                 4.15462881]),
          <BarContainer object of 10 artists>)
```

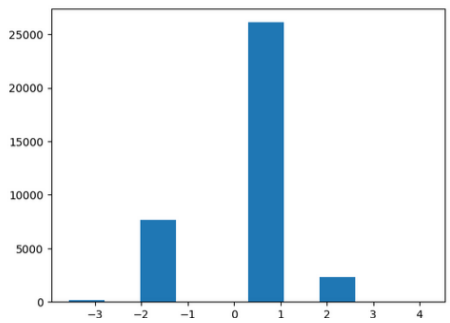

Figure 17: Outlier Check 1

```
In [32]: #no_of_children
         ##Outlier assessment via histogram
         hotel_data['Z_Score_no_of_children']=stats.zscore(hotel_data['no_of_children'])
         hotel_data[['no_of_children', 'Z_Score_no_of_children']].head

Out[32]: <bound method NDFrame.head of        no_of_children  Z_Score_no_of_children
         0                   0               -0.26147
         1                   0               -0.26147
         2                   0               -0.26147
         3                   0               -0.26147
         4                   0               -0.26147
         ...               ...                    ...
         36270               0               -0.26147
         36271               0               -0.26147
         36272               0               -0.26147
         36273               0               -0.26147
         36274               0               -0.26147

         [36275 rows x 2 columns]>

In [33]: plt.hist(hotel_data['Z_Score_no_of_children'])

Out[33]: (array([3.3577e+04, 1.6180e+03, 1.0580e+03, 1.9000e+01, 0.0000e+00,
                 0.0000e+00, 0.0000e+00, 0.0000e+00, 2.0000e+00, 1.0000e+00]),
          array([-0.26147045,  2.22212223,  4.70571492,  7.1893076 ,  9.67290028,
                 12.15649296, 14.64008564, 17.12367832, 19.607271  , 22.09086369,
                 24.57445637]),
          <BarContainer object of 10 artists>)
```

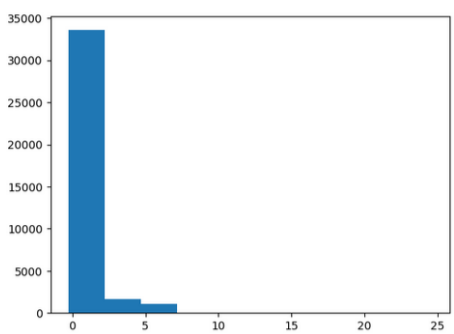

Figure 18: Outlier Check 2

```
In [34]: #no_of_weekend_nights
         ##Outlier assessment via histogram
         hotel_data['Z_Score_no_of_weekend_nights']=stats.zscore(hotel_data['no_of_weekend_nights'])
         hotel_data[['no_of_weekend_nights', 'Z_Score_no_of_weekend_nights']].head

Out[34]: <bound method NDFrame.head of        no_of_weekend_nights  Z_Score_no_of_weekend_nights
         0                       1                    0.217401
         1                       2                    1.365993
         2                       2                    1.365993
         3                       0                   -0.931190
         4                       1                    0.217401
         ...                   ...                         ...
         36270                   2                    1.365993
         36271                   1                    0.217401
         36272                   2                    1.365993
         36273                   0                   -0.931190
         36274                   1                    0.217401

         [36275 rows x 2 columns]>

In [35]: plt.hist(hotel_data['Z_Score_no_of_weekend_nights'])

Out[35]: (array([1.6872e+04, 9.9950e+03, 9.0710e+03, 0.0000e+00, 1.5300e+02,
                 1.2900e+02, 0.0000e+00, 3.4000e+01, 2.0000e+01, 1.0000e+00]),
          array([-0.93119021, -0.12717622,  0.67683777,  1.48085176,  2.28486576,
                 3.08887975,  3.89289374,  4.69690773,  5.50092173,  6.30493572,
                 7.10894971]),
          <BarContainer object of 10 artists>)
```

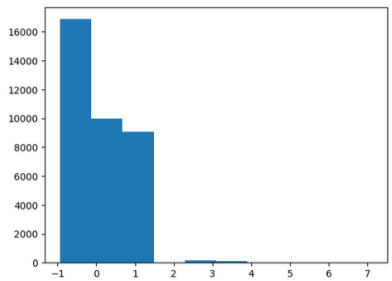

Figure 19: Outlier Check 3

```
In [36]: #no_of_week_nights
         ##Outlier assessment via histogram
         hotel_data['Z_Score_no_of_week_nights']=stats.zscore(hotel_data['no_of_week_nights'])
         hotel_data[['no_of_week_nights', 'Z_Score_no_of_week_nights']].head

Out[36]: <bound method NDFrame.head of        no_of_week_nights  Z_Score_no_of_week_nights
         0                       2                   -0.144803
         1                       3                    0.563972
         2                       1                   -0.853578
         3                       2                   -0.144803
         4                       1                   -0.853578
         ...                   ...                         ...
         36270                   6                    2.690296
         36271                   3                    0.563972
         36272                   6                    2.690296
         36273                   3                    0.563972
         36274                   2                   -0.144803

         [36275 rows x 2 columns]>

In [37]: plt.hist(hotel_data['Z_Score_no_of_week_nights'])

Out[37]: (array([1.1875e+04, 1.9283e+04, 4.6040e+03, 1.8900e+02, 1.7500e+02,
                 9.6000e+01, 1.7000e+01, 1.4000e+01, 1.7000e+01, 5.0000e+00]),
          array([-1.5623526 , -0.35743547,  0.84748167,  2.0523988 ,  3.25731593,
                 4.46223307,  5.6671502 ,  6.87206733,  8.07698447,  9.2819016 ,
                10.48681873]),
          <BarContainer object of 10 artists>)
```

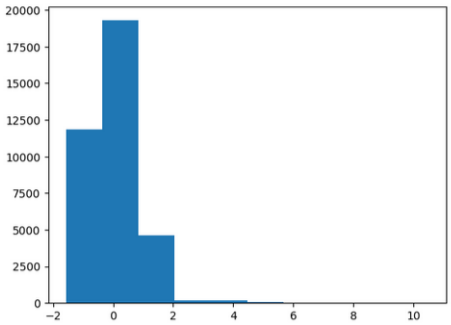

Figure 20: Outlier Check 4

```
In [38]: #lead_time
         ##Outlier assessment via histogram
         hotel_data['Z_Score_lead_time']=stats.zscore(hotel_data['lead_time'])
         hotel_data[['lead_time', 'Z_Score_lead_time']].head

Out[38]: <bound method NDFrame.head of        lead_time  Z_Score_lead_time
         0            224           1.614896
         1              5          -0.933701
         2              1          -0.980250
         3            211           1.463610
         4             48          -0.433291
         ...          ...                ...
         36270         85          -0.002706
         36271        228           1.661446
         36272        148           0.730452
         36273         63          -0.258730
         36274        207           1.417060

         [36275 rows x 2 columns]>

In [39]: plt.hist(hotel_data['Z_Score_lead_time'])

Out[39]: (array([15803.,  7213.,  4655.,  3080.,  2253.,  1209.,  1314.,   483.,
                  163.,   102.]),
          array([-0.99188783, -0.47634973,  0.03918838,  0.55472649,  1.0702646 ,
                 1.5858027 ,  2.10134081,  2.61687892,  3.13241703,  3.64795513,
                 4.16349324]),
          <BarContainer object of 10 artists>)
```

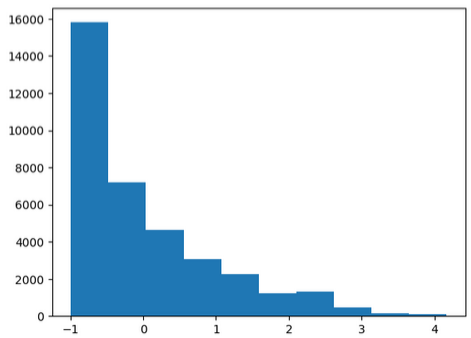

Figure 21: Outlier Check 5

```
In [40]: #no_of_previous_cancellations
         ##Outlier assessment via histogram
         hotel_data['Z_Score_no_of_previous_cancellations']=stats.zscore(hotel_data['no_of_previous_cancellations'])
         hotel_data[['no_of_previous_cancellations', 'Z_Score_no_of_previous_cancellations']].head

Out[40]: <bound method NDFrame.head of        no_of_previous_cancellations  Z_Score_no_of_previous_cancellations
         0                                   0                             -0.063393
         1                                   0                             -0.063393
         2                                   0                             -0.063393
         3                                   0                             -0.063393
         4                                   0                             -0.063393
         ...                               ...                                   ...
         36270                               0                             -0.063393
         36271                               0                             -0.063393
         36272                               0                             -0.063393
         36273                               0                             -0.063393
         36274                               0                             -0.063393

         [36275 rows x 2 columns]>

In [41]: plt.hist(hotel_data['Z_Score_no_of_previous_cancellations'])

Out[41]: (array([3.6135e+04, 4.6000e+01, 4.3000e+01, 2.1000e+01, 1.0000e+00,
                 0.0000e+00, 0.0000e+00, 0.0000e+00, 2.5000e+01, 4.0000e+00]),
          array([-0.06339327,  3.46608521,  6.99556369, 10.52504217, 14.05452065,
                 17.58399913, 21.11347761, 24.64295609, 28.17243457, 31.70191305,
                 35.23139153]),
          <BarContainer object of 10 artists>)
```

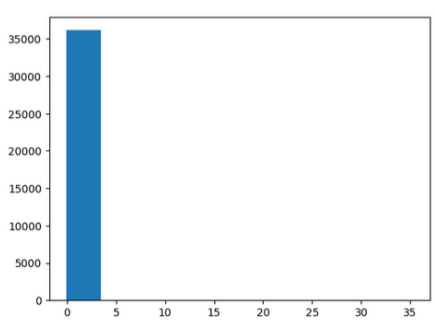

Figure 22: Outlier Check 6

```
In [42]: #no_of_previous_bookings_not_canceled
         ##Outlier assessment via histogram
         hotel_data['Z_Score_no_of_previous_bookings_not_canceled']=stats.zscore(hotel_data['no_of_previous_bookings_not_canceled'])
         hotel_data[['no_of_previous_bookings_not_canceled', 'Z_Score_no_of_previous_bookings_not_canceled']].head

Out[42]: <bound method NDFrame.head of        no_of_previous_bookings_not_canceled  \
         0                                       0
         1                                       0
         2                                       0
         3                                       0
         4                                       0
         ...                                   ...
         36270                                   0
         36271                                   0
         36272                                   0
         36273                                   0
         36274                                   0

               Z_Score_no_of_previous_bookings_not_canceled
         0                                       -0.087456
         1                                       -0.087456
         2                                       -0.087456
         3                                       -0.087456
         4                                       -0.087456
         ...                                           ...
         36270                                   -0.087456
         36271                                   -0.087456
         36272                                   -0.087456
         36273                                   -0.087456
         36274                                   -0.087456

         [36275 rows x 2 columns]>

In [43]: plt.hist(hotel_data['Z_Score_no_of_previous_bookings_not_canceled'])

Out[43]: (array([3.6008e+04, 1.3600e+02, 4.9000e+01, 3.3000e+01, 1.3000e+01,
                 1.0000e+01, 6.0000e+00, 7.0000e+00, 7.0000e+00, 6.0000e+00]),
          array([-0.08745646,  3.2189948 ,  6.52544605,  9.8318973 , 13.13834855,
                 16.44479981, 19.75125106, 23.05770231, 26.36415357, 29.67060482,
                 32.97705607]),
          <BarContainer object of 10 artists>)
```

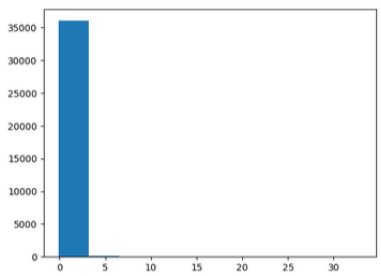

Figure 23: Outlier Check 7

```
In [44]: #avg_price_per_room
         ##Outlier assessment via histogram
         hotel_data['Z_Score_avg_price_per_room']=stats.zscore(hotel_data['avg_price_per_room'])
         hotel_data[['avg_price_per_room', 'Z_Score_avg_price_per_room']].head

Out[44]: <bound method NDFrame.head of        avg_price_per_room  Z_Score_avg_price_per_room
         0                   65.00                   -1.095033
         1                  106.68                    0.092806
         2                   60.00                   -1.237528
         3                  100.00                   -0.097567
         4                   94.50                   -0.254312
         ...                   ...                         ...
         36270              167.80                    1.834665
         36271               90.95                   -0.355483
         36272               98.39                   -0.143451
         36273               94.50                   -0.254312
         36274              161.67                    1.659966

         [36275 rows x 2 columns]>

In [45]: plt.hist(hotel_data['Z_Score_avg_price_per_room'])

Out[45]: (array([1.0810e+03, 2.0645e+04, 1.2570e+04, 1.6970e+03, 2.5100e+02,
                 2.6000e+01, 4.0000e+00, 0.0000e+00, 0.0000e+00, 1.0000e+00]),
          array([-2.94746829, -1.40852185,  0.1304246 ,  1.66937104,  3.20831748,
                 4.74726393,  6.28621037,  7.82515682,  9.36410326, 10.9030497 ,
                12.44199615]),
          <BarContainer object of 10 artists>)
```

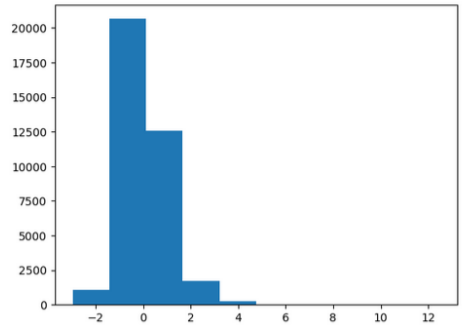

Figure 24: Outlier Check 8

```
In [46]: #no_of_special_requests
         ##Outlier assessment via histogram
         hotel_data['Z_Score_no_of_special_requests']=stats.zscore(hotel_data['no_of_special_requests'])
         hotel_data[['no_of_special_requests', 'Z_Score_no_of_special_requests']].head

Out[46]: <bound method NDFrame.head of        no_of_special_requests  Z_Score_no_of_special_requests
         0                           0                      -0.788140
         1                           1                       0.483760
         2                           0                      -0.788140
         3                           0                      -0.788140
         4                           0                      -0.788140
         ...                       ...                            ...
         36270                       1                       0.483760
         36271                       2                       1.755661
         36272                       2                       1.755661
         36273                       0                      -0.788140
         36274                       0                      -0.788140

         [36275 rows x 2 columns]>

In [47]: plt.hist(hotel_data['Z_Score_no_of_special_requests'])

Out[47]: (array([1.9777e+04, 0.0000e+00, 1.1373e+04, 4.3640e+03, 0.0000e+00,
                 0.0000e+00, 6.7500e+02, 0.0000e+00, 7.8000e+01, 8.0000e+00]),
          array([-0.78813999, -0.15218977,  0.48376045,  1.11971067,  1.75566089,
                  2.39161111,  3.02756133,  3.66351155,  4.29946177,  4.93541199,
                  5.5713622 ]),
          <BarContainer object of 10 artists>)
```

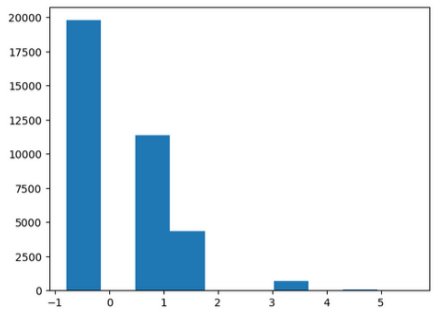

Figure 25: Outlier Check 9

```
In [48]: #length_of_stay
         ##Outlier assessment via histogram
         hotel_data['Z_Score_length_of_stay']=stats.zscore(hotel_data['length_of_stay'])
         hotel_data[['length_of_stay', 'Z_Score_length_of_stay']].head

Out[48]: <bound method NDFrame.head of        length_of_stay  Z_Score_length_of_stay
         0                   3               -0.008412
         1                   5                1.111414
         2                   3               -0.008412
         3                   2               -0.568325
         4                   2               -0.568325
         ...               ...                     ...
         36270               8                2.791152
         36271               4                0.551501
         36272               8                2.791152
         36273               3               -0.008412
         36274               3               -0.008412

         [36275 rows x 2 columns]>

In [49]: plt.hist(hotel_data['Z_Score_length_of_stay'])

Out[49]: (array([1.5154e+04, 1.5945e+04, 4.5930e+03, 2.9000e+02, 1.4800e+02,
                 7.4000e+01, 3.7000e+01, 1.4000e+01, 1.5000e+01, 5.0000e+00]),
          array([-1.68815098, -0.34435996,  0.99943107,  2.34322209,  3.68701311,
                  5.03080413,  6.37459516,  7.71838618,  9.0621772 , 10.40596822,
                 11.74975924]),
          <BarContainer object of 10 artists>)
```

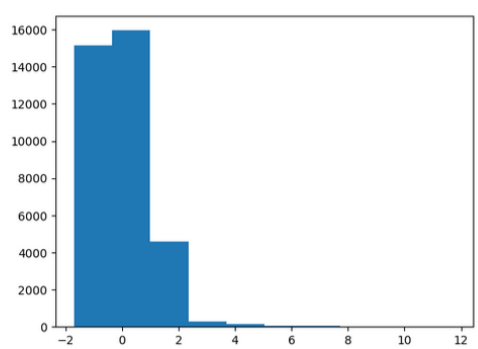

Figure 26: Outlier Check 10

```
In [50]: #count_of_guests
         ##Outlier assessment via histogram
         hotel_data['Z_Score_count_of_guests']=stats.zscore(hotel_data['count_of_guests'])
         hotel_data[['count_of_guests', 'Z_Score_count_of_guests']].head

Out[50]: <bound method NDFrame.head of        count_of_guests  Z_Score_count_of_guests
         0                     2                 0.076515
         1                     2                 0.076515
         2                     1                -1.461195
         3                     2                 0.076515
         4                     2                 0.076515
         ...                 ...                      ...
         36270                 3                 1.614224
         36271                 2                 0.076515
         36272                 2                 0.076515
         36273                 2                 0.076515
         36274                 2                 0.076515

         [36275 rows x 2 columns]>

In [51]: plt.hist(hotel_data['Z_Score_count_of_guests'])

Out[51]: (array([3.1494e+04, 3.8510e+03, 9.1200e+02, 1.5000e+01, 0.0000e+00,
                 0.0000e+00, 0.0000e+00, 0.0000e+00, 1.0000e+00, 2.0000e+00]),
          array([-1.46119479,  0.23028549,  1.92176576,  3.61324603,  5.3047263 ,
                  6.99620657,  8.68768684, 10.37916711, 12.07064738, 13.76212765,
                 15.45360792]),
          <BarContainer object of 10 artists>)
```

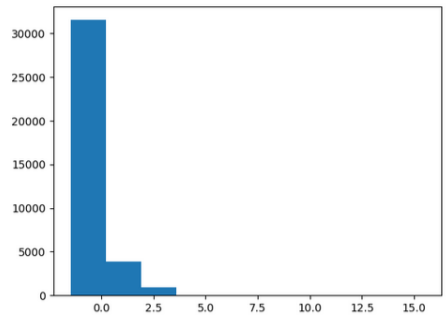

Figure 27: Outlier Check 11

```
In [52]: #count_of_previous_bookings
         ##Outlier assessment via histogram
         hotel_data['Z_Score_count_of_previous_bookings']=stats.zscore(hotel_data['count_of_previous_bookings'])
         hotel_data[['count_of_previous_bookings', 'Z_Score_count_of_previous_bookings']].head

Out[52]: <bound method NDFrame.head of        count_of_previous_bookings  Z_Score_count_of_previous_bookings
         0                                   0                           -0.090467
         1                                   0                           -0.090467
         2                                   0                           -0.090467
         3                                   0                           -0.090467
         4                                   0                           -0.090467
         ...                               ...                                 ...
         36270                               0                           -0.090467
         36271                               0                           -0.090467
         36272                               0                           -0.090467
         36273                               0                           -0.090467
         36274                               0                           -0.090467

         [36275 rows x 2 columns]>

In [53]: plt.hist(hotel_data['Z_Score_count_of_previous_bookings'])

Out[53]: (array([3.6004e+04, 1.2600e+02, 6.2000e+01, 3.0000e+01, 1.9000e+01,
                 9.0000e+00, 6.0000e+00, 7.0000e+00, 6.0000e+00, 6.0000e+00]),
          array([-0.09046676,  3.1850734 ,  6.46061357,  9.73615374, 13.0116939 ,
                 16.28723407, 19.56277424, 22.8383144 , 26.11385457, 29.38939473,
                 32.6649349 ]),
          <BarContainer object of 10 artists>)
```

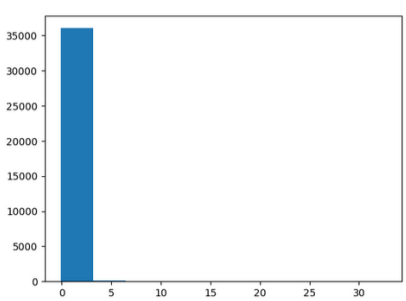

Figure 28: Outlier Check 12

Figures 17 through 28 represent checks for outliers for the quantitative variables considered for use in the logistic regression model. Outliers have the potential to impact the

performance of the logistic regression model that we will later develop. The treatment of any outliers is an important aspect of the data preparation process.

GeeksForGeeks (2024) notes that "One of the techniques used to handle the outliers is to remove them from the dataset. However, removing outliers can potentially lead to the loss of valuable data. In such cases where the outliers are allocated to represent the valid data points, it may be appropriate to leave them unchanged." Based on the outlier values indicated above in the Z-score histograms, as well as the values noted in the assessment of the value counts for the variables, all of the data points appear to be valid. As such, outliers will not be treated before performing any analysis.

```
In [54]: #Bivariate visualizations for all numerical variables using booking_status as target variable

         ##no_of_adults
         ct_no_of_adults = pd.crosstab(hotel_data['no_of_adults'], hotel_data['booking_status'])
         ct_no_of_adults.plot(kind='bar', stacked=True)
         plt.ylabel('Count')
         plt.title('Stacked Bar Chart of Booking Status by Number of Adults')
         plt.legend(["No Cancel", "Cancel"])
         plt.show()
```

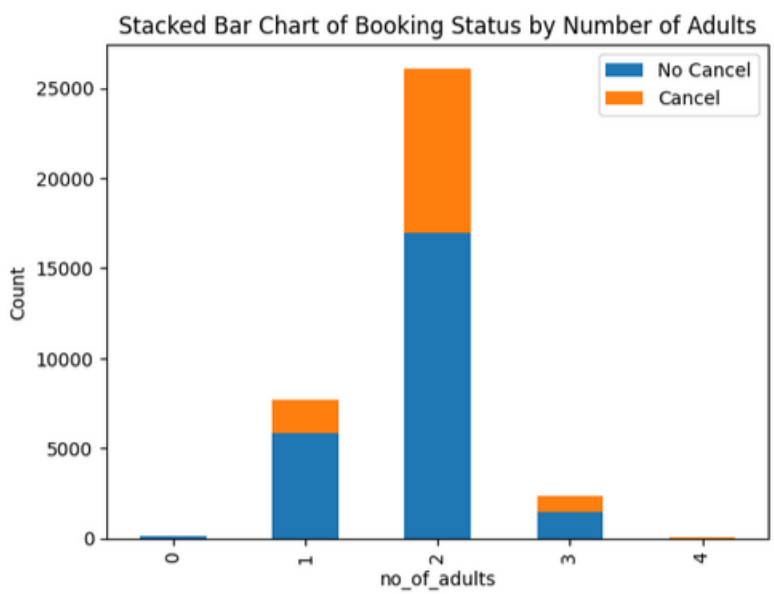

Figure 29: Bivariate Visualization 1

The next step in the data preparation process is to analyze the bivariate visualizations between the potential independent variables and the target variable of booking status. Any differences in cancelation rates between the unique values for a variable can potentially indicate

that this variable will have influence over the cancelation rate and could be useful in the logistic regression model. Above, we can see that reservations with two adults appear to have a higher cancelation rate than reservations with greater or fewer than two adults.

```
In [55]: ##no_of_children
         ct_no_of_children = pd.crosstab(hotel_data['no_of_children'], hotel_data['booking_status'])
         ct_no_of_children.plot(kind='bar', stacked=True)
         plt.ylabel('Count')
         plt.title('Stacked Bar Chart of Booking Status by Number of Children')
         plt.legend(["No Cancel", "Cancel"])
         plt.show()
```

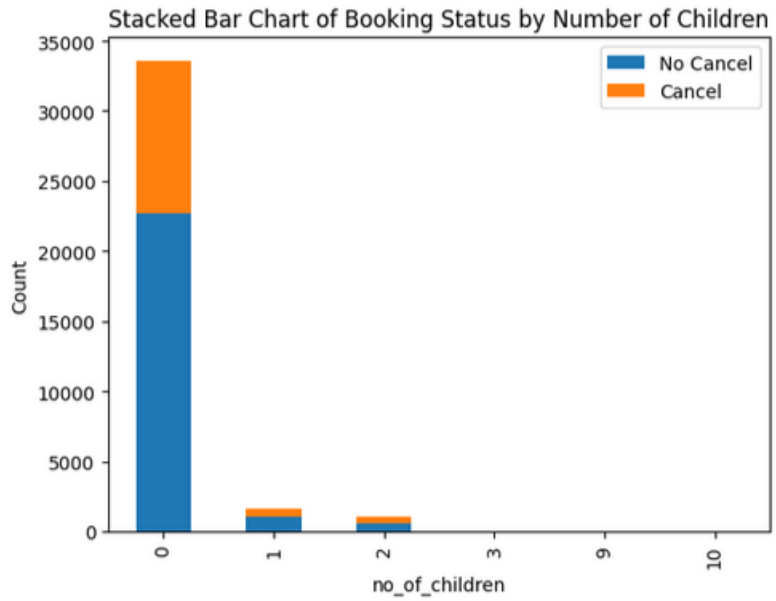

Figure 30: Bivariate Visualization 2

In the above visualization, we can see that there does not appear to be a significant difference in cancelation rate based solely on the number of children on a reservation. It is also reinforced that an overwhelming majority of the reservations include zero children.

```
In [56]: ##no_of_weekend_nights
         ct_no_of_weekend_nights = pd.crosstab(hotel_data['no_of_weekend_nights'], hotel_data['booking_status'])
         ct_no_of_weekend_nights.plot(kind='bar', stacked=True)
         plt.ylabel('Count')
         plt.title('Stacked Bar Chart of Booking Status by Number of Weekend Nights')
         plt.legend(["No Cancel", "Cancel"])
         plt.show()
```

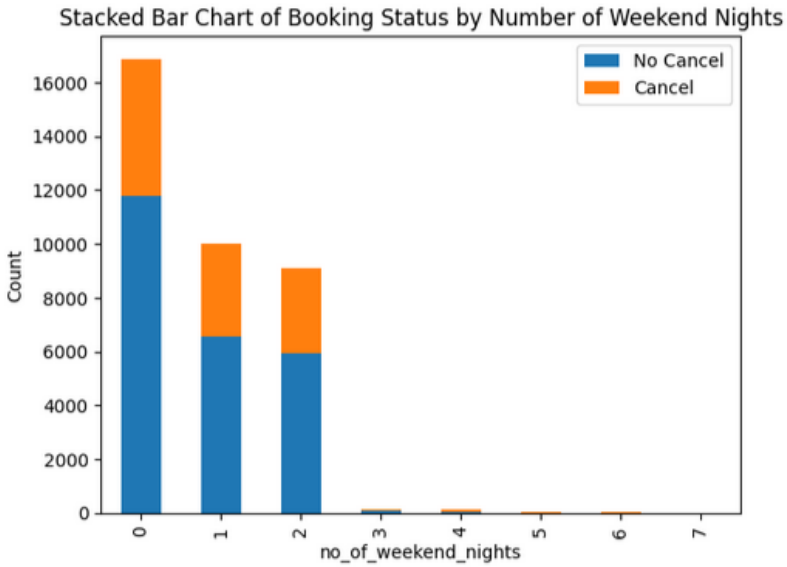

Figure 31: Bivariate Visualization 3

In the above visualization, we can see that there does not appear to be any significant difference in cancelation rate based solely on the number of weekend nights for a reservation.

```
In [57]: ##no_of_week_nights
ct_no_of_week_nights = pd.crosstab(hotel_data['no_of_week_nights'], hotel_data['booking_status'])
ct_no_of_week_nights.plot(kind='bar', stacked=True)
plt.ylabel('Count')
plt.title('Stacked Bar Chart of Booking Status by Number of Week Nights')
plt.legend(["No Cancel", "Cancel"])
plt.show()
```

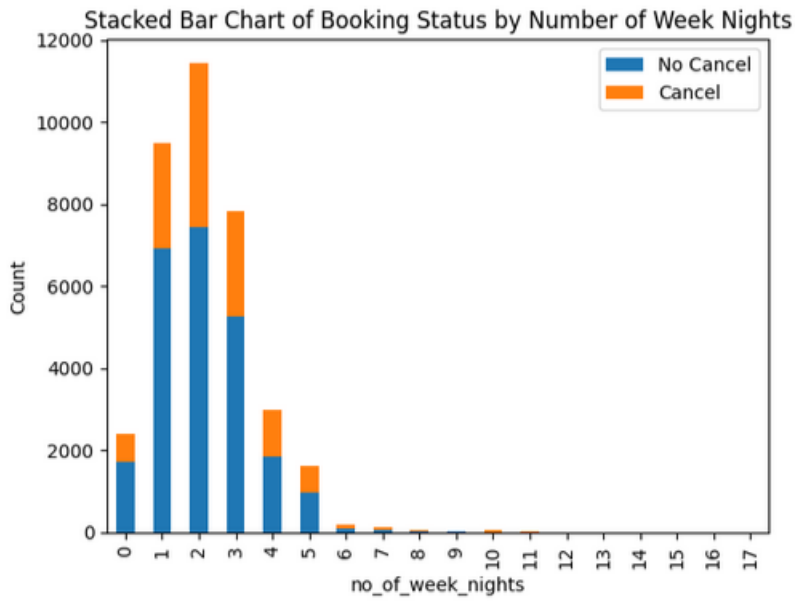

Figure 32: Bivariate Visualization 4

In the above visualization, we can see that there may potentially be differences in the cancelation rate based on the number of week nights. This potential difference may warrant further analysis of this variable.

```
In [58]: ##required_car_parking_space
         ct_required_car_parking_space = pd.crosstab(hotel_data['required_car_parking_space'], hotel_data['booking_status'])
         ct_required_car_parking_space.plot(kind='bar', stacked=True)
         plt.ylabel('Count')
         plt.title('Stacked Bar Chart of Booking Status by Required Car Parking Space')
         plt.legend(["No Cancel", "Cancel"])
         plt.show()
```

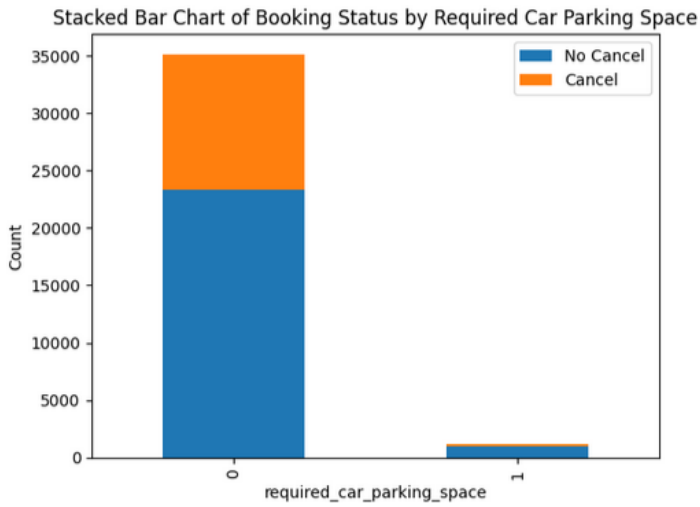

Figure 33: Bivariate Visualization 5

The above visualization displays a potentially significant difference in cancelation rate based on whether the reservation required a parking space. This makes intuitive sense as it would be likely that if a guest requires a parking space, they may have also booked a rental car. This level of increased investment could potentially be a factor in decreasing the likelihood that a reservation gets canceled.

```
In [59]: ##lead_time - put these into buckets below and see if that shows anything different
         ct_lead_time = pd.crosstab(hotel_data['lead_time'], hotel_data['booking_status'])
         ct_lead_time.plot(kind='bar', stacked=True)
         plt.ylabel('Count')
         plt.title('Stacked Bar Chart of Booking Status by Lead Time')
         plt.legend(["No Cancel", "Cancel"])
         plt.show()
```

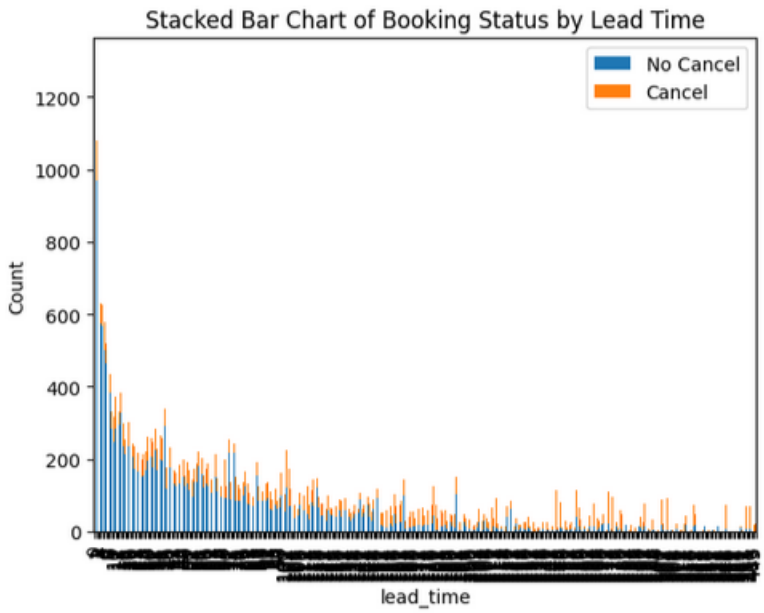

Figure 34: Bivariate Visualization 6

The above visualization presents a difficult-to-understand picture, and indicates that if any value is to be derived from the Lead Time variable, this variable likely needs to be modified into a format that is easier to interpret. This modification is performed after the conclusion of the initial bivariate visualization analysis.

```
In [60]: ##arrival_year
         ct_arrival_year = pd.crosstab(hotel_data['arrival_year'], hotel_data['booking_status'])
         ct_arrival_year.plot(kind='bar', stacked=True)
         plt.ylabel('Count')
         plt.title('Stacked Bar Chart of Booking Status by Arrival Year')
         plt.legend(["No Cancel", "Cancel"])
         plt.show()
```

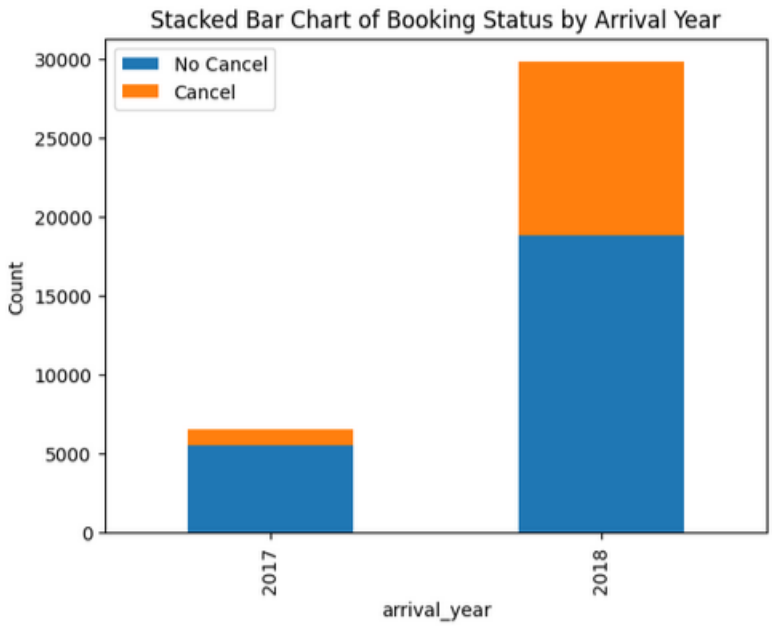

Figure 35: Bivariate Visualization 7

Figure 35 above indicates that there may be a significant difference in terms of cancelation rate based on whether the reservation was for 2017 or 2018. This could indicate that there is a benefit to further analysis.

```
In [61]: ##arrival_month
         ct_arrival_month = pd.crosstab(hotel_data['arrival_month'], hotel_data['booking_status'])
         ct_arrival_month.plot(kind='bar', stacked=True)
         plt.ylabel('Count')
         plt.title('Stacked Bar Chart of Booking Status by Arrival Month')
         plt.legend(["No Cancel", "Cancel"])
         plt.show()
```

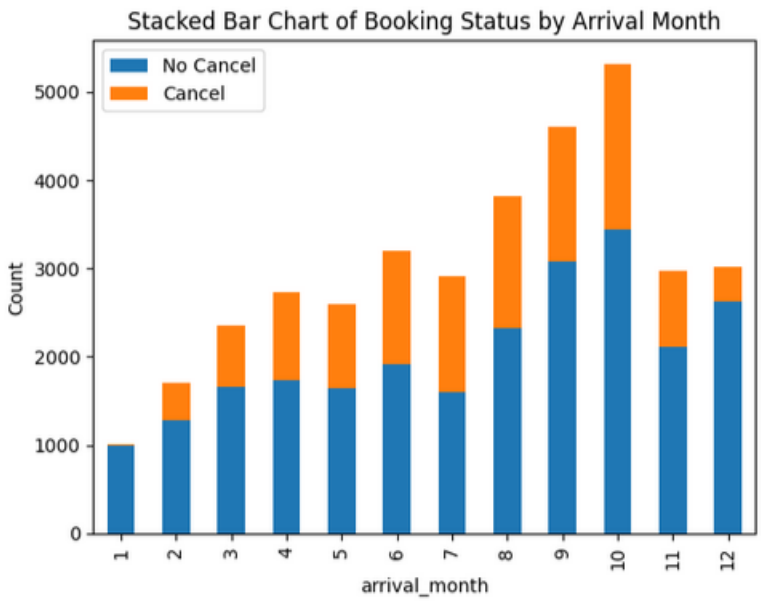

Figure 36: Bivariate Visualization 8

Above, it can be seen that there is a higher rate of cancelation in the summer and fall months relative to the winter. This difference could indicate some potential effect that the month of arrival has on cancelation status, which can be assessed when developing the initial logistic regression model.

```
In [62]: ##arrival_date
ct_arrival_date = pd.crosstab(hotel_data['arrival_date'], hotel_data['booking_status'])
ct_arrival_date.plot(kind='bar', stacked=True)
plt.ylabel('Count')
plt.title('Stacked Bar Chart of Booking Status by Arrival Date')
plt.legend(["No Cancel", "Cancel"])
plt.show()
```

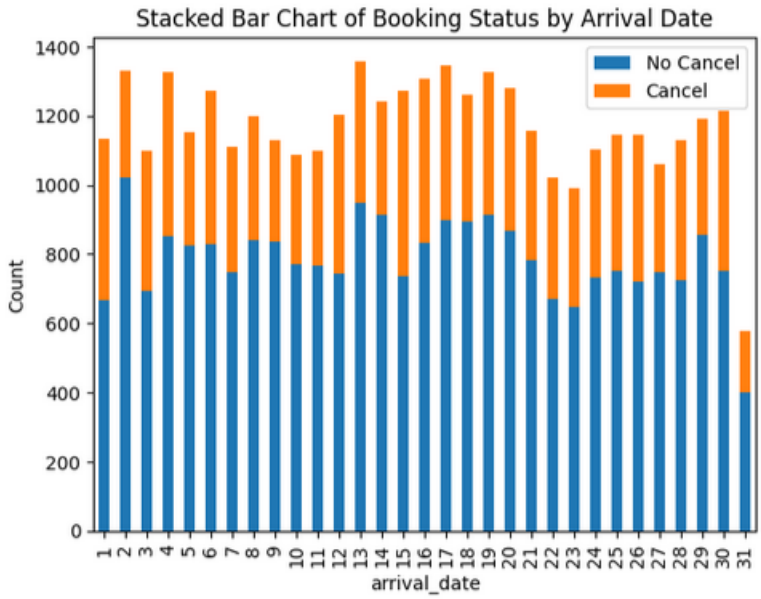

Figure 37: Bivariate Visualization 9

The above distribution shows that while there may potentially be a difference in cancelation rate by date, there may be too many distinct values to parse out any meaningful relationship between the date of arrival and whether the reservation is canceled.

```
In [63]: ##repeated_guest
         ct_repeated_guest = pd.crosstab(hotel_data['repeated_guest'], hotel_data['booking_status'])
         ct_repeated_guest.plot(kind='bar', stacked=True)
         plt.ylabel('Count')
         plt.title('Stacked Bar Chart of Booking Status by Repeated Guest Status')
         plt.legend(["No Cancel", "Cancel"])
         plt.show()
```

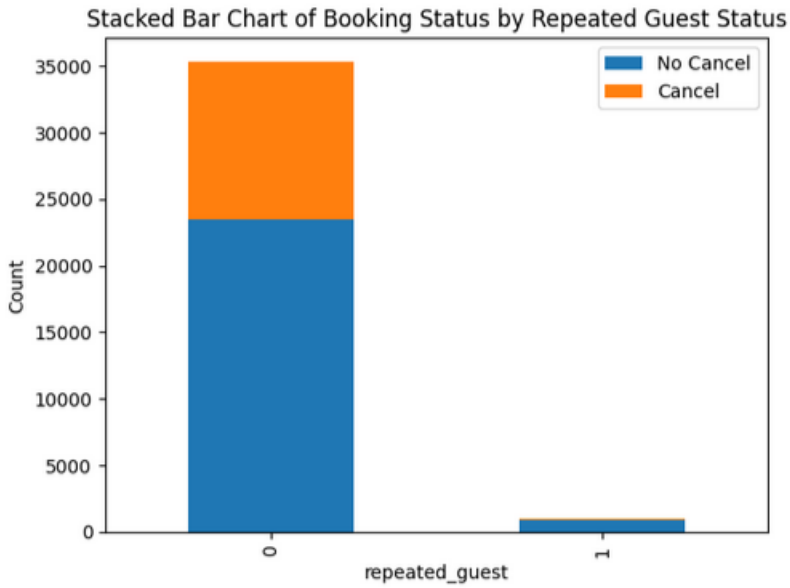

Figure 38: Bivariate Visualization 10

Above, it can be seen that there is a significantly higher cancelation rate among guests who have not stayed at the hotel before. This is a valuable piece of information that makes sense intrinsically, as guests who have stayed at the hotel before logically would be less likely to cancel their next reservation. Based solely on the above visualization, it seems likely that repeated guest status will have an influence on cancelation likelihood in the logistic regression model.

```
In [64]: ##no_of_previous_cancellations
         ct_no_of_previous_cancellations = pd.crosstab(hotel_data['no_of_previous_cancellations'], hotel_data['booking_status']
         ct_no_of_previous_cancellations.plot(kind='bar', stacked=True)
         plt.ylabel('Count')
         plt.title('Stacked Bar Chart of Booking Status by Number of Previous Cancellations')
         plt.legend(["No Cancel", "Cancel"])
         plt.show()
```

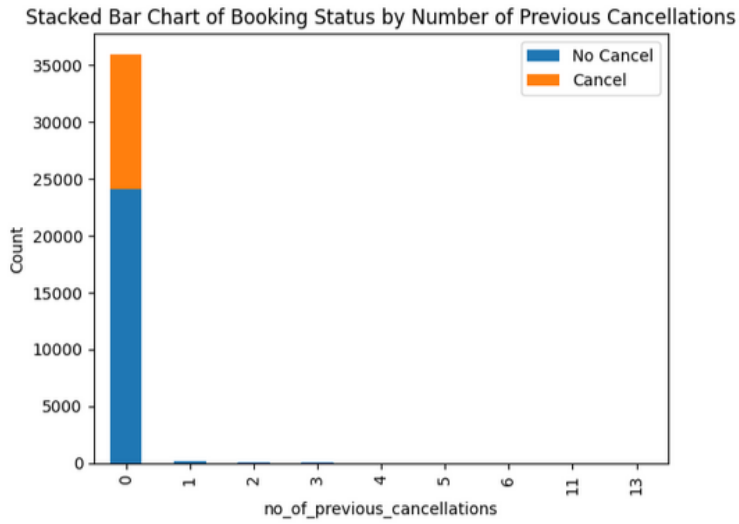

Figure 39: Bivariate Visualization 11

The above visualization demonstrates that almost all reservations do not include a previous cancelation. Based on this, it would seem difficult to include this variable in the logistic regression model in such a way that would determine any meaningful influence on cancelation status.

```
In [65]: ##no_of_previous_bookings_not_cancelled
         ct_no_of_previous_bookings_not_canceled = pd.crosstab(hotel_data['no_of_previous_bookings_not_canceled'], hotel_data['
         ct_no_of_previous_bookings_not_canceled.plot(kind='bar', stacked=True)
         plt.ylabel('Count')
         plt.title('Stacked Bar Chart of Booking Status by Number of Previous Bookings Not Cancelled')
         plt.legend(["No Cancel", "Cancel"])
         plt.show()
```

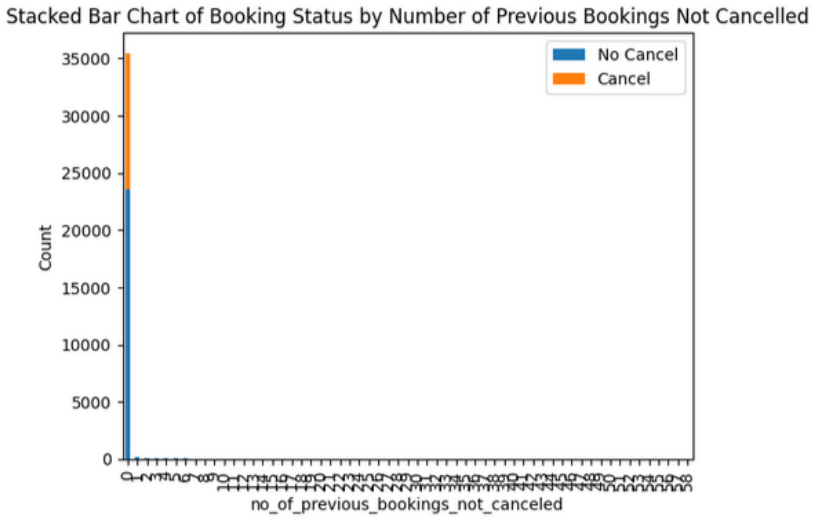

Figure 40: Bivariate Visualization 12

Figure 40 shows a difficult-to-interpret relationship between the number of previous uncanceled reservations and whether the current reservation is canceled. The number of distinct counts for this variable indicates that, as-is, this variable will struggle to be used in the model.

```
In [66]: ##avg_price_per_room - put into buckets similar to lead time
         ct_avg_price_per_room = pd.crosstab(hotel_data['avg_price_per_room'], hotel_data['booking_status'])
         ct_avg_price_per_room.plot(kind='bar', stacked=True)
         plt.ylabel('Count')
         plt.title('Stacked Bar Chart of Booking Status by Average Price Per Room')
         plt.legend(["No Cancel", "Cancel"])
         plt.show()
```

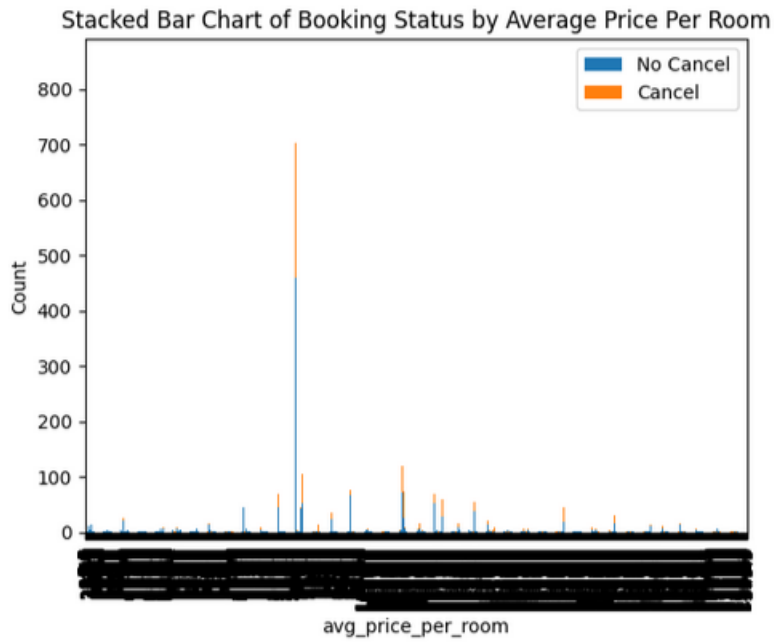

Figure 41: Bivariate Visualization 13

Figure 41 falls victim to the same issue as Figure 40 due to the count of unique values for this variable. However, this can and will be mitigated later in the data preparation stage by converting the variable into High and Low buckets based on the average value for the variable.

```
In [67]: ##no_of_special_requests
         ct_no_of_special_requests = pd.crosstab(hotel_data['no_of_special_requests'], hotel_data['booking_status'])
         ct_no_of_special_requests.plot(kind='bar', stacked=True)
         plt.ylabel('Count')
         plt.title('Stacked Bar Chart of Booking Status by Number of Special Requests')
         plt.legend(["No Cancel", "Cancel"])
         plt.show()
```

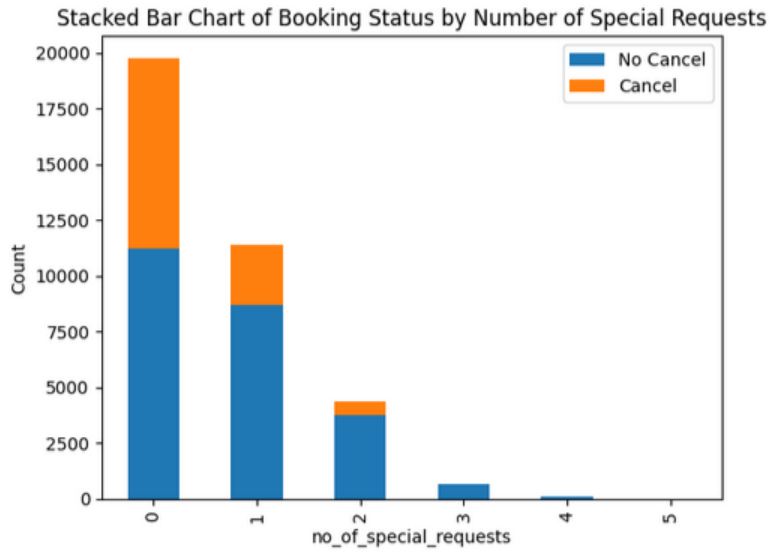

Figure 42: Bivariate Visualization 14

Figure 42 notes a significant difference in cancelation rate between reservations with special requests and reservations without. This indicates that special requests likely have an influence on whether the reservation is canceled.

```
In [68]: ##length_of_stay - put into buckets as well
         ct_length_of_stay = pd.crosstab(hotel_data['length_of_stay'], hotel_data['booking_status'])
         ct_length_of_stay.plot(kind='bar', stacked=True)
         plt.ylabel('Count')
         plt.title('Stacked Bar Chart of Booking Status by Length of Stay')
         plt.legend(["No Cancel", "Cancel"])
         plt.show()
```

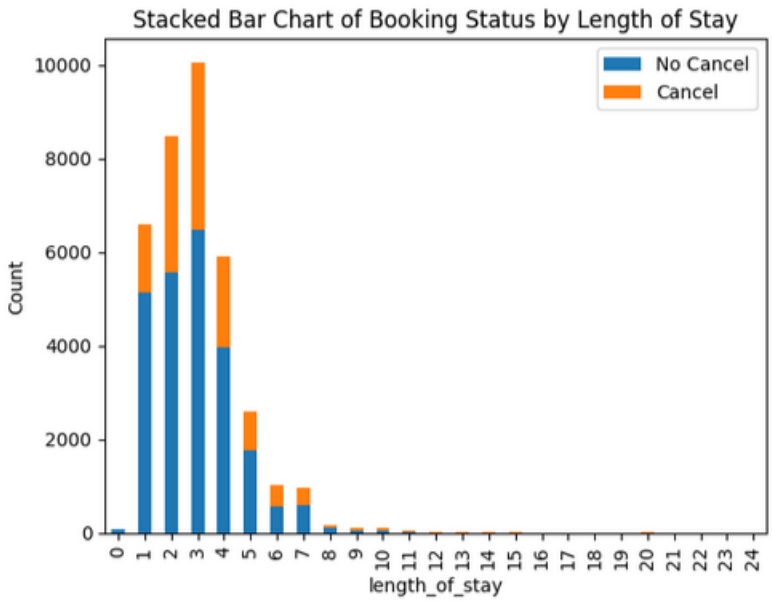

Figure 43: Bivariate Visualization 15

Similar to the price per room in Figure 41, the length of stay for a reservation can be put into High and Low buckets in order to potentially parse out any influence that this variable as on cancelation status. As-is, it appears difficult to assess any influence simply by looking at the distribution.

```
In [69]: ##count_of_guests
         ct_count_of_guests = pd.crosstab(hotel_data['count_of_guests'], hotel_data['booking_status'])
         ct_count_of_guests.plot(kind='bar', stacked=True)
         plt.ylabel('Count')
         plt.title('Stacked Bar Chart of Booking Status by Count of Guests')
         plt.legend(["No Cancel", "Cancel"])
         plt.show()
```

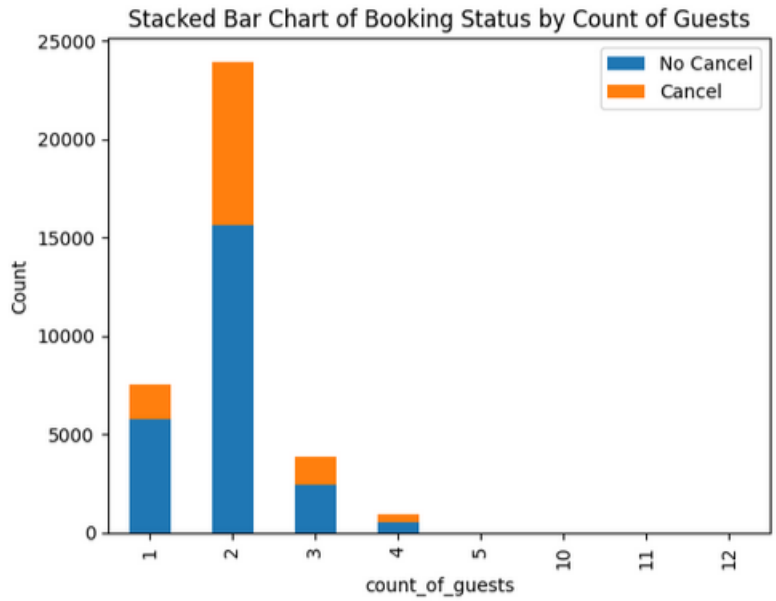

Figure 44: Bivariate Visualization 16

Based on the above, it would appear that reservations with two guests are canceled at a higher rate. However, it will be important to handle multicollinearity between this variable and the variables it is derived from. Once that is handled, there may be a relationship between the number of guests and cancelation status.

```
In [70]: ##count_of_previous_bookings
         ct_count_of_previous_bookings = pd.crosstab(hotel_data['count_of_previous_bookings'], hotel_data['booking_status'])
         ct_count_of_previous_bookings.plot(kind='bar', stacked=True)
         plt.ylabel('Count')
         plt.title('Stacked Bar Chart of Booking Status by Count of Previous Bookings')
         plt.legend(["No Cancel", "Cancel"])
         plt.show()
```

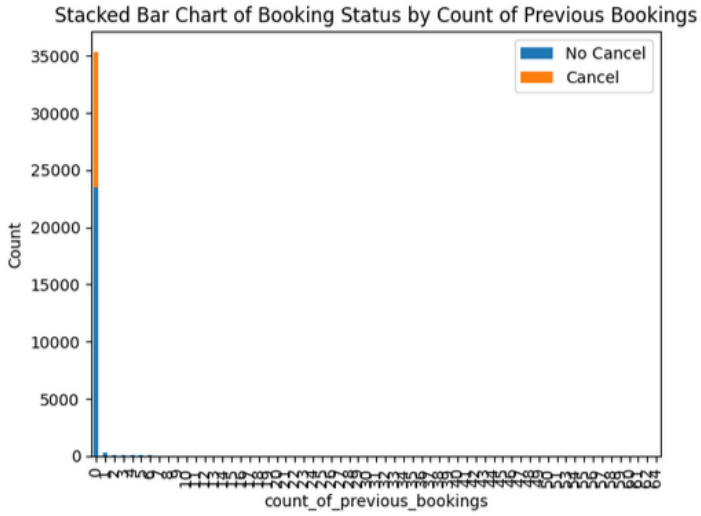

Figure 45: Bivariate Visualization 17

Unlike in Figure 44, it does not appear that any useful information can be gleaned from this derived variable. As the majority of guests are first time visitors, it is unlikely that the number of previous bookings will have an impact on cancelation status.

```
In [71]: ##prior_cancellation
         ct_prior_cancellation = pd.crosstab(hotel_data['prior_cancellation'], hotel_data['booking_status'])
         ct_prior_cancellation.plot(kind='bar', stacked=True)
         plt.ylabel('Count')
         plt.title('Stacked Bar Chart of Booking Status by Prior Cancellation Status')
         plt.legend(["No Cancel", "Cancel"])
         plt.show()
```

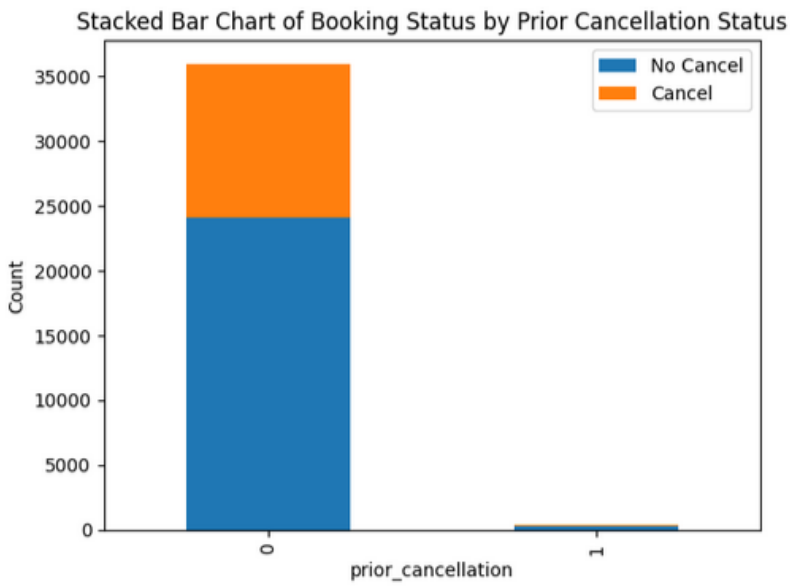

Figure 46: Bivariate Visualization 18

Similarly to Figure 45, due to the imbalance between the values for this variable, it would seem difficult to draw any conclusions for cancelation status based specifically on prior cancellation status.

```
In [72]: ##special_request_flag
         ct_special_request_flag = pd.crosstab(hotel_data['special_request_flag'], hotel_data['booking_status'])
         ct_special_request_flag.plot(kind='bar', stacked=True)
         plt.ylabel('Count')
         plt.title('Stacked Bar Chart of Booking Status by Special Request Status')
         plt.legend(["No Cancel", "Cancel"])
         plt.show()
```

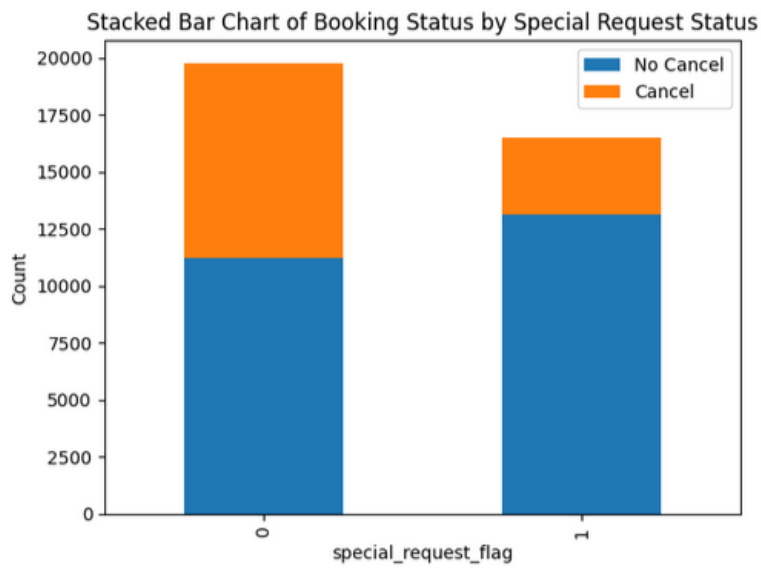

Figure 47: Bivariate Visualization 19

Unlike the previous two derived variables, Special Request Status appears to show a significant difference between reservations with no special request and reservations with a special request in terms of cancelation rate. Reservations with a special request appear to cancel at a significantly lower rate. The simplicity of the variable values also indicates that it could be useful in the logistic regression model due to its relative ease of interpretability.

```
In [73]: #Put lead_time, avg_price_per_room, length_of_stay into Low, Medium, High buckets based on mean or median values,
         #then do another histogram for each to parse out more useful data. Could also just do low/high.

         ##lead_time - skewed so make buckets based on median
         hotel_data['lead_time'].mean()
Out[73]: 85.23255685733976

In [74]: hotel_data['lead_time'].median()
Out[74]: 57.0

In [75]: ##avg_price_per_room - not skewed so make buckets based on mean
         hotel_data['avg_price_per_room'].mean()
Out[75]: 103.42353907649897

In [76]: hotel_data['avg_price_per_room'].median()
Out[76]: 99.45

In [77]: ##length_of_stay - not skewed so make buckets based on mean
         hotel_data['length_of_stay'].mean()
Out[77]: 3.0150241212956583

In [78]: hotel_data['length_of_stay'].median()
Out[78]: 3.0
```

Figure 48: Buckets Creation

As Lead Time, Price per Room, and Length of Stay had distributions that were difficult to interpret, it makes sense to convert these variables into binary variables based on whether the value of the variable is considered ‘High’ or ‘Low’. Values above average can be considered high, whereas values below average can be considered low. One determination that is needed is whether the average value used will be the mean or the median. In relatively symmetrical, non-skewed data, the mean and the median will be roughly equal.

Above, we can see that Average Price per Room and Length of Stay have roughly equal mean and median, indicating that we can use the average based on the mean. However, Lead Time has a significantly higher mean than median, indicating that the values are skewed. Penn State (n.d.) states, “For distributions that have outliers or are skewed, the median is often the preferred measure of central tendency because the median is more resistant to outliers than the mean.” As such, the median will be used as the average for the soon-to-be-created Lead Time buckets.

```
In [79]: ##lead time bucket with bivariate visualization
median_lead_time = hotel_data['lead_time'].median()
hotel_data['lead_time_buckets'] = hotel_data['lead_time'].apply(lambda x: 'Low' if x <= median_lead_time else 'High')

ct_lead_time_buckets = pd.crosstab(hotel_data['lead_time_buckets'], hotel_data['booking_status'])
ct_lead_time_buckets.plot(kind = 'bar', stacked=True)
plt.ylabel('Count')
plt.title('Stacked Bar Chart of Booking Status by High or Low Lead Time')
plt.legend(["No Cancel", "Cancel"])
plt.show()
```

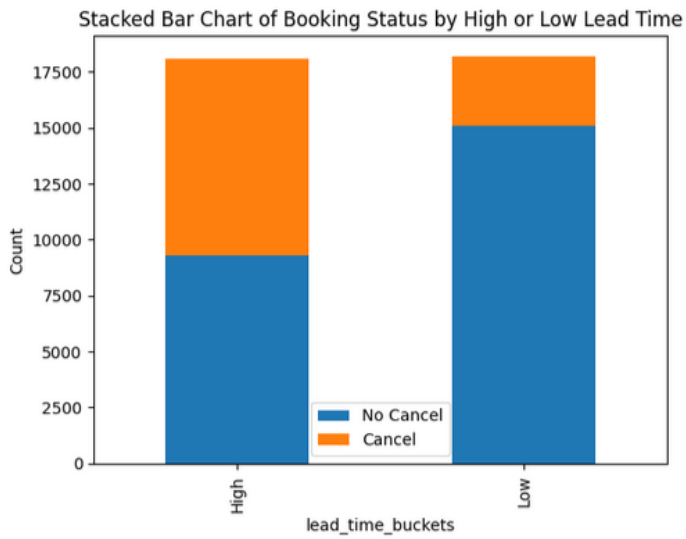

Figure 49: Bivariate Visualization 20

Above, Lead Time is split into High and Low buckets. A new bivariate visualization is created, showing a significant difference in cancelation rate between reservations booked relatively far out and reservations booked relatively close to the date of the hotel stay. This indicates that this variable could be significant in the logistic regression model.

```
In [80]: ##avg price per room bucket with bivariate visualization
mean_avg_price_per_room = hotel_data['avg_price_per_room'].mean()
hotel_data['avg_price_per_room_buckets'] = hotel_data['avg_price_per_room'].apply(lambda x: 'Low' if x <= mean_avg_pri

ct_avg_price_per_room_buckets = pd.crosstab(hotel_data['avg_price_per_room_buckets'], hotel_data['booking_status'])
ct_avg_price_per_room_buckets.plot(kind = 'bar', stacked=True)
plt.ylabel('Count')
plt.title('Stacked Bar Chart of Booking Status by High or Low Average Price Per Room')
plt.legend(["No Cancel", "Cancel"])
plt.show()
```

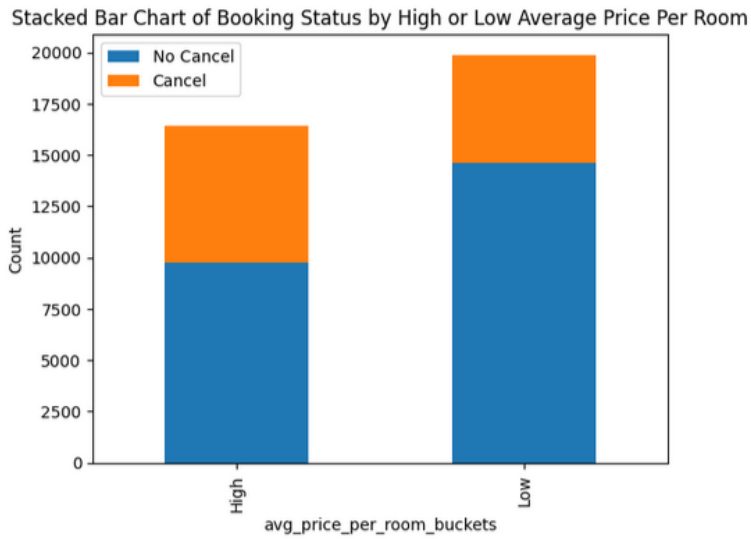

Figure 50: Bivariate Visualization 21

Above, buckets with High and Low labels are created for average price per room. We can see that there appears to be a difference between cancelation rate, with higher-cost rooms being canceled at a higher rate than lower-cost rooms. Whether this difference is significant enough to impact the target variable will remain to be seen.

```
In [81]: ##length of stay bucket with bivariate visualization
mean_length_of_stay = hotel_data['length_of_stay'].mean()
hotel_data['length_of_stay_buckets'] = hotel_data['length_of_stay'].apply(lambda x: 'Low' if x <= mean_length_of_stay

ct_mean_length_of_stay_buckets = pd.crosstab(hotel_data['length_of_stay_buckets'], hotel_data['booking_status'])
ct_mean_length_of_stay_buckets.plot(kind = 'bar', stacked=True)
plt.ylabel('Count')
plt.title('Stacked Bar Chart of Booking Status by High or Low Average Length of Stay')
plt.legend(["No Cancel", "Cancel"])
plt.show()
```

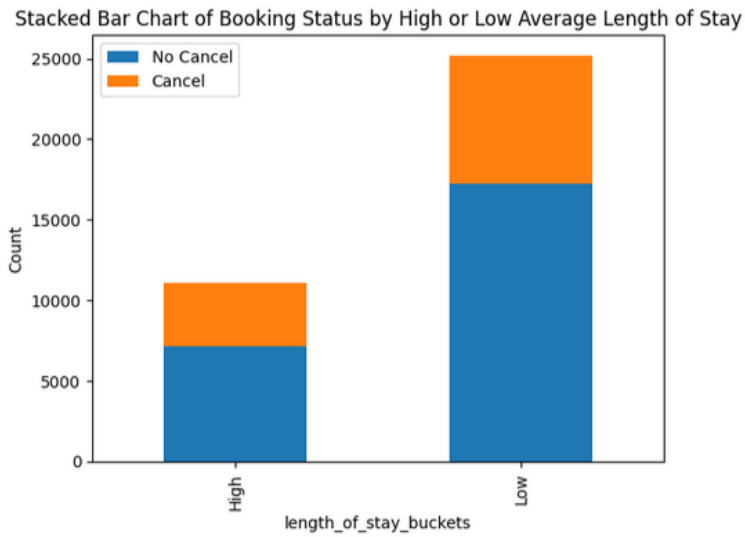

Figure 51: Bivariate Visualization 22

Figure 51 shows a difference in cancelation rate based on whether the length of stay was High or Low. This makes sense, as reservations with high lengths represent a greater investment than reservations with shorter lengths. In essence, canceling a reservation with a greater number of days represents a more impactful decision by the guest than canceling a reservation with a fewer number of days.

```
In [82]: #Check value counts for categorical variables to see if they can/should be re-encoded
         ##'type_of_meal_plan', 'room_type_reserved', 'market_segment_type'
         print(hotel_data['type_of_meal_plan'].value_counts()) #Don't have dictionary info of which type means what,
         #but can split it into "Selected a Meal Plan" versus "Did not select a meal plan"

         type_of_meal_plan
         Meal Plan 1     27835
         Not Selected     5130
         Meal Plan 2      3305
         Meal Plan 3         5
         Name: count, dtype: int64

In [83]: print(hotel_data['room_type_reserved'].value_counts()) #Don't have dictionary info of which type means what

         room_type_reserved
         Room_Type 1    28130
         Room_Type 4     6057
         Room_Type 6      966
         Room_Type 2      692
         Room_Type 5      265
         Room_Type 7      158
         Room_Type 3        7
         Name: count, dtype: int64

In [84]: print(hotel_data['market_segment_type'].value_counts()) #Online 1, Other 0

         market_segment_type
         Online           23214
         Offline          10528
         Corporate         2017
         Complementary      391
         Aviation           125
         Name: count, dtype: int64
```

Figure 52: Value Counts for Re-encoding

Figure 52 represents a look at the value counts for categorical variables that can potentially be re-encoded to numeric datatypes. Although there is no data dictionary, the type of meal plan can be re-encoded to whether or not the reservation selected a meal plan. The selection of a meal plan represents an investment by the potential guest, with the logic being that any sort of investment is likely to reduce the likelihood of cancelation. Additionally, the market segment type can be binary re-encoded to either be an online reservation or other.

The type of room reserved is a variable that falls victim to not having a data dictionary for explanation. While it may be tempting to potentially use a Likert scale to convert this variable to numeric, there is no guarantee that the increase in the room type number represents a more valuable type of room. As such, this variable will not be numerically encoded and will not be included as part of the regression analysis.

```
In [85]: #Categorical variable re-expression via one-hot encoding - room_type_reserved not included since no dictionary
         ##type_of_meal_plan (Selected Meal Plan: 1, Did not Select Meal Plan: 0)
         hotel_data['type_of_meal_plan'].unique()

Out[85]: array(['Meal Plan 1', 'Not Selected', 'Meal Plan 2', 'Meal Plan 3'],
               dtype=object)

In [86]: hotel_data['selected_meal_plan'] = hotel_data['type_of_meal_plan'].replace({"Meal Plan 1": 1, "Meal Plan 2": 1, "Meal
         hotel_data['selected_meal_plan'].value_counts()

         C:\Users\matth\AppData\Local\Temp\ipykernel_12488\2570540558.py:1: FutureWarning: Downcasting behavior in `replace` i
         s deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer_objec
         ts(copy=False)`. To opt-in to the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`
           hotel_data['selected_meal_plan'] = hotel_data['type_of_meal_plan'].replace({"Meal Plan 1": 1, "Meal Plan 2": 1, "Me
         al Plan 3": 1, "Not Selected": 0})

Out[86]: selected_meal_plan
         1    31145
         0     5130
         Name: count, dtype: int64

In [87]: ##market_segment_type (Online: 1, Other 0)
         hotel_data['market_segment_type'].unique()

Out[87]: array(['Offline', 'Online', 'Corporate', 'Aviation', 'Complementary'],
               dtype=object)

In [88]: hotel_data['online_reservation'] = hotel_data['market_segment_type'].replace({"Online": 1, "Offline": 0, "Corporate":
         hotel_data['online_reservation'].value_counts()

         C:\Users\matth\AppData\Local\Temp\ipykernel_12488\920506920.py:1: FutureWarning: Downcasting behavior in `replace` is
         deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer_objects
         (copy=False)`. To opt-in to the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`
           hotel_data['online_reservation'] = hotel_data['market_segment_type'].replace({"Online": 1, "Offline": 0, "Corporate
         ": 0, "Aviation": 0, "Complementary": 0})

Out[88]: online_reservation
         1    23214
         0    13061
         Name: count, dtype: int64
```

Figure 53: Categorical Re-expression

Figure 53 represents the act of re-expressing the noted variables from Figure 52 into numeric data types. We can see that a majority of reservations selected a meal plan, and a majority of reservations were made online. Now that these categorical variables have been re-expressed as numeric, they can potentially be used as part of the logistic regression model.

```
In [89]: ##Convert remaining objects to numerical for use in logistic regression
         hotel_data['lead_time_buckets'] = hotel_data['lead_time_buckets'].map({'Low': 0, 'High': 1})
         hotel_data['avg_price_per_room_buckets'] = hotel_data['avg_price_per_room_buckets'].map({'Low': 0, 'High': 1})
         hotel_data['length_of_stay_buckets'] = hotel_data['length_of_stay_buckets'].map({'Low': 0, 'High': 1})

In [90]: hotel_data['lead_time_buckets'].value_counts()

Out[90]: lead_time_buckets
         0    18202
         1    18073
         Name: count, dtype: int64

In [91]: hotel_data['avg_price_per_room_buckets'].value_counts()

Out[91]: avg_price_per_room_buckets
         0    19876
         1    16399
         Name: count, dtype: int64

In [92]: hotel_data['length_of_stay_buckets'].value_counts()

Out[92]: length_of_stay_buckets
         0    25206
         1    11069
         Name: count, dtype: int64
```

Figure 54: Convert Buckets to Numeric

The final step of the data preparation process is to convert the created High and Low buckets for the above variables into numeric datatypes, with 1 for High and 0 for Low. This allows for these variables to be used as independent variables in the logistic regression model. Above, we can see relatively equal values between High and Low lead times, while the Low values are greater for price per room and length of stay.

One specific tool used in the data preparation process was the creation of newly derived variables based on previously existing variables. The advantage of this specific tool is that the derived variables can more easily display differences in the impact the variable has on the outcome variable relative to the initial variables. For example, the derived lead time buckets noted in Figures 48 and 53 are easier to interpret than the initial lead time variable.

One disadvantage of creating derived variable is that it leads to multicollinearity between variables. According to Allison (2024), “Multicollinearity is a common problem when estimating linear or generalized linear models, including logistic regression and Cox regression. It occurs when there are high correlations among predictor variables, leading to unreliable and unstable estimates of regression coefficients.” As the lead time buckets contain data from the initial lead time variable, these variables are going to be highly (if not entirely) correlated and they cannot both be included as independent variables in the logistic regression model. This element of multicollinearity will be addressed and handled in the Analysis portion of this report.

D. Analysis

Once the data has been sufficiently extracted and prepared for analysis, the initial logistic regression model can be constructed.

```
In [93]: #Construct initial logistic regression model
         print(hotel_data.dtypes)

Booking_ID                                        object
no_of_adults                                       int64
no_of_children                                     int64
no_of_weekend_nights                               int64
no_of_week_nights                                  int64
type_of_meal_plan                                 object
required_car_parking_space                         int64
room_type_reserved                                object
lead_time                                          int64
arrival_year                                       int64
arrival_month                                      int64
arrival_date                                       int64
market_segment_type                               object
repeated_guest                                     int64
no_of_previous_cancellations                       int64
no_of_previous_bookings_not_canceled               int64
avg_price_per_room                               float64
no_of_special_requests                             int64
booking_status                                     int64
length_of_stay                                     int64
count_of_guests                                    int64
count_of_previous_bookings                         int64
prior_cancellation                                 int64
special_request_flag                               int64
Z_Score_no_of_adults                             float64
Z_Score_no_of_children                           float64
Z_Score_no_of_weekend_nights                     float64
Z_Score_no_of_week_nights                        float64
Z_Score_lead_time                                float64
Z_Score_no_of_previous_cancellations             float64
Z_Score_no_of_previous_bookings_not_canceled     float64
Z_Score_avg_price_per_room                       float64
Z_Score_no_of_special_requests                   float64
Z_Score_length_of_stay                           float64
Z_Score_count_of_guests                          float64
Z_Score_count_of_previous_bookings               float64
lead_time_buckets                                  int64
avg_price_per_room_buckets                         int64
length_of_stay_buckets                             int64
selected_meal_plan                                 int64
online_reservation                                 int64
dtype: object
```

Figure 55: Datatypes Before Regression

Prior to the construction of the initial logistic regression, it is helpful to look at the datatypes of the variables that can be used as independent variables in the model. Above, we can see that any numeric datatype could theoretically be included in the initial model.

```
In [94]: independent_variables = ['no_of_adults', 'no_of_children', 'no_of_weekend_nights', 'no_of_week_nights', 'required_car_
                                  'arrival_year', 'arrival_month', 'arrival_date', 'repeated_guest', 'no_of_previous_cancellati
                                  'no_of_previous_bookings_not_canceled', 'no_of_special_requests',
                                  'count_of_guests', 'count_of_previous_bookings', 'prior_cancellation', 'special_request_flag'
                                  'online_reservation', 'selected_meal_plan', 'lead_time_buckets', 'avg_price_per_room_buckets'
                                  'length_of_stay_buckets']
         X = hotel_data[independent_variables]
         y = hotel_data['booking_status']

         X = sm.add_constant(X)
         initial_model = sm.Logit(y, X)
         initial_result = initial_model.fit()
         print(initial_result.summary())

         Warning: Maximum number of iterations has been exceeded.
                  Current function value: 0.472862
                  Iterations: 35
                                  Logit Regression Results
         ==============================================================================
         Dep. Variable:         booking_status   No. Observations:                36275
         Model:                          Logit   Df Residuals:                    36254
         Method:                           MLE   Df Model:                           20
         Date:                Wed, 08 Jan 2025   Pseudo R-squ.:                  0.2524
         Time:                        11:59:41   Log-Likelihood:                -17153.
         converged:                       False   LL-Null:                       -22944.
         Covariance Type:            nonrobust   LLR p-value:                     0.000
         ==========================================================================================
                                             coef    std err          z      P>|z|      [0.025      0.975]
         ------------------------------------------------------------------------------------------
         const                            -2126.4971     88.931    -23.912      0.000   -2300.799   -1952.195
         no_of_adults                         0.0669   1.66e+05   4.03e-07      1.000   -3.26e+05    3.26e+05
         no_of_children                       0.0399   1.66e+05    2.4e-07      1.000   -3.26e+05    3.26e+05
         no_of_weekend_nights                 0.1789      0.018     10.094      0.000       0.144       0.214
         no_of_week_nights                    0.1410      0.013     11.193      0.000       0.116       0.166
         required_car_parking_space          -1.4610      0.114    -12.808      0.000      -1.685      -1.237
         arrival_year                         1.0524      0.044     23.882      0.000       0.966       1.139
         arrival_month                        0.0353      0.005      7.367      0.000       0.026       0.045
         arrival_date                         0.0037      0.002      2.452      0.014       0.001       0.007
         repeated_guest                     -21.8344   9282.223     -0.002      0.998   -1.82e+04    1.82e+04
         no_of_previous_cancellations         0.1285        nan        nan        nan         nan         nan
         no_of_previous_bookings_not_canceled -0.1373        nan        nan        nan         nan         nan
         no_of_special_requests              -1.1251      0.048    -23.650      0.000      -1.218      -1.032
         count_of_guests                      0.1066   1.66e+05   6.42e-07      1.000   -3.26e+05    3.26e+05
         count_of_previous_bookings          -0.0089        nan        nan        nan         nan         nan
         prior_cancellation                  20.9466   9282.223      0.002      0.998   -1.82e+04    1.82e+04
         special_request_flag                -0.4348      0.065     -6.724      0.000      -0.562      -0.308
         online_reservation                   1.1720      0.035     33.805      0.000       1.104       1.240
         selected_meal_plan                  -0.0551      0.041     -1.338      0.181      -0.136       0.026
         lead_time_buckets                    1.7009      0.030     56.971      0.000       1.642       1.759
         avg_price_per_room_buckets           0.7572      0.030     25.523      0.000       0.699       0.815
         length_of_stay_buckets              -0.5238      0.043    -12.050      0.000      -0.609      -0.439
         ==========================================================================================
```

Figure 56: Initial Regression

When we create the initial regression model above, we can see that a lot of the values are not what were expected. Notably, the results of the logistic regression did not converge. Filho (2023) indicates that "One common reason is the presence of highly correlated features in your dataset. This can make it difficult for the logistic regression model to determine the relationship between each feature and the target variable, leading to non-convergence." In the Data Preparation section, we noted that multicollinearity was likely to be an issue in the model initially due to new variables being derived from existing variables.

```
In [95]: #Correlation matrix
         hotel_data_for_corr = hotel_data[['booking_status', 'no_of_adults', 'no_of_children', 'no_of_weekend_nights', 'no_of_week_n
                                           'arrival_year', 'arrival_month', 'arrival_date', 'repeated_guest', 'no_of_previous_cancellations',
                                           'no_of_previous_bookings_not_canceled', 'no_of_special_requests',
                                           'count_of_guests', 'count_of_previous_bookings', 'prior_cancellation', 'special_request_flag',
                                           'online_reservation', 'selected_meal_plan', 'lead_time_buckets', 'avg_price_per_room_buckets',
                                           'length_of_stay_buckets']]
         corr_matrix = hotel_data_for_corr.corr()
         plt.figure(figsize=(16, 12))
         sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f")
         plt.show()
```

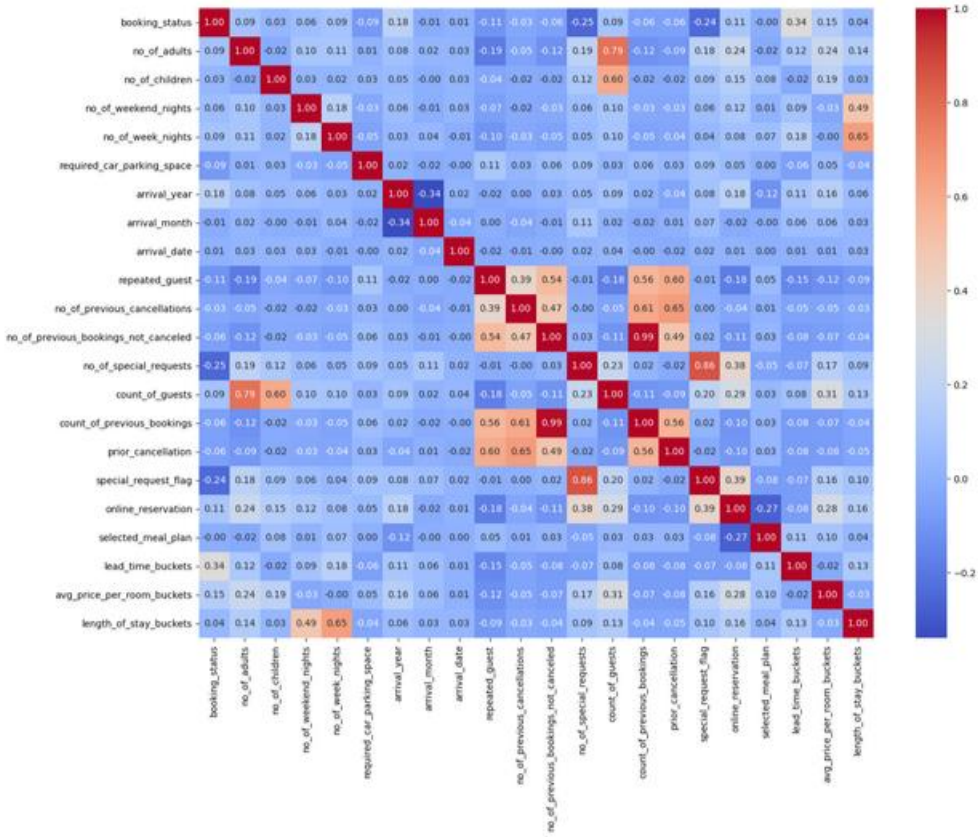

Figure 57: Correlation Matrix

Solving the issue of multicollinearity initially begins by creating a correlation matrix from the dependent variable and independent variables. We can see via the heatmap that there are a number of variables that are very strongly correlated.

```
In [96]: ##Table of the highest correlated values (anything over 0.5?)
corr_pairs = corr_matrix.unstack().reset_index()
corr_pairs.columns = ['Variable 1', 'Variable 2', 'Correlation']

high_corr = corr_pairs[(corr_pairs['Correlation'] > 0.5) & (corr_pairs['Variable 1'] != corr_pairs['Variable 2'])]
high_corr = high_corr.sort_values(by='Correlation', ascending=False)
high_corr['Correlation'] = high_corr['Correlation'].round(2)  # Limit to 2 decimal places


fig, ax = plt.subplots(figsize=(10, len(high_corr) * 0.4))  # Adjust height based on rows
ax.axis('tight')
ax.axis('off')

table = ax.table(
    cellText=high_corr.values,  # Data to display
    colLabels=high_corr.columns,  # Column headers
    loc='center',  # Table position
)

table.auto_set_font_size(False)
table.set_fontsize(10)
table.auto_set_column_width(col=list(range(len(high_corr.columns))))  # Adjust column widths

plt.title("Highly Correlated Variables", fontsize=12)
plt.show()
```

Highly Correlated Variables

Variable 1	Variable 2	Correlation
no_of_previous_bookings_not_canceled	count_of_previous_bookings	0.99
count_of_previous_bookings	no_of_previous_bookings_not_canceled	0.99
special_request_flag	no_of_special_requests	0.86
no_of_special_requests	special_request_flag	0.86
no_of_adults	count_of_guests	0.79
count_of_guests	no_of_adults	0.79
no_of_previous_cancellations	prior_cancellation	0.65
prior_cancellation	no_of_previous_cancellations	0.65
no_of_week_nights	length_of_stay_buckets	0.65
length_of_stay_buckets	no_of_week_nights	0.65
no_of_previous_cancellations	count_of_previous_bookings	0.61
count_of_previous_bookings	no_of_previous_cancellations	0.61
no_of_children	count_of_guests	0.6
count_of_guests	no_of_children	0.6
repeated_guest	prior_cancellation	0.6
prior_cancellation	repeated_guest	0.6
count_of_previous_bookings	prior_cancellation	0.56
prior_cancellation	count_of_previous_bookings	0.56
repeated_guest	count_of_previous_bookings	0.56
count_of_previous_bookings	repeated_guest	0.56
no_of_previous_bookings_not_canceled	repeated_guest	0.54
repeated_guest	no_of_previous_bookings_not_canceled	0.54

Figure 58: Correlation Table

The above table represents another look at the most correlated variables, presented in a way that can help identify whether these variables represent multicollinearity or are simply highly correlated. We can see that among the most correlated variables are the variables that we derived from existing variables. As such, we can address this issue by removing several of the variables before attempting an initial logistic regression again.

```
In [97]: #Remove highly correlated variables (we know some of them are multicollinear because we derived some columns from othe
         #Then run the logit model again as our new model
         #Dropped variables will be the ones that showed less valuable info in the bivariate visualizations than their derived
         ##Going the drop the following: 'no_of_previous_bookings_not_canceled', 'no_of_week_nights', 'no_of_special_requests',
         ## 'no_of_previous_cancellations', 'no_of_children', 'no_of_weekend_nights', 'no_of_adults'
         independent_variables = ['required_car_parking_space','arrival_year', 'arrival_month', 'arrival_date', 'repeated_guest
                                  'count_of_guests', 'count_of_previous_bookings', 'prior_cancellation', 'special_request_flag'
                                  'online_reservation', 'selected_meal_plan', 'lead_time_buckets', 'avg_price_per_room_buckets'
                                  'length_of_stay_buckets']
         X = hotel_data[independent_variables]
         y = hotel_data['booking_status']

         X = sm.add_constant(X)
         model1 = sm.Logit(y, X)
         result1 = model1.fit()
         print(result1.summary())

         Warning: Maximum number of iterations has been exceeded.
                  Current function value: 0.485106
                  Iterations: 35
                                   Logit Regression Results
         ==============================================================================
         Dep. Variable:         booking_status   No. Observations:                36275
         Model:                          Logit   Df Residuals:                    36260
         Method:                           MLE   Df Model:                           14
         Date:                Wed, 08 Jan 2025   Pseudo R-squ.:                  0.2330
         Time:                        12:00:04   Log-Likelihood:                -17597.
         converged:                       False   LL-Null:                       -22944.
         Covariance Type:            nonrobust   LLR p-value:                     0.000
         ==========================================================================================
                                      coef    std err          z      P>|z|      [0.025      0.975]
         ------------------------------------------------------------------------------------------
         const                   -2120.1619     88.035    -24.083      0.000   -2292.706   -1947.617
         required_car_parking_space   -1.4438      0.112    -12.896      0.000      -1.663      -1.224
         arrival_year              1.0496      0.044     24.060      0.000       0.964       1.135
         arrival_month             0.0232      0.005      4.920      0.000       0.014       0.032
         arrival_date              0.0028      0.001      1.876      0.061      -0.000       0.006
         repeated_guest          -17.2907    797.061     -0.022      0.983   -1579.502    1544.921
         count_of_guests           0.1161      0.022      5.218      0.000       0.072       0.160
         count_of_previous_bookings   -0.0410      0.035     -1.163      0.245      -0.110       0.028
         prior_cancellation       16.5386    797.061      0.021      0.983   -1545.673    1578.750
         special_request_flag     -1.8430      0.031    -58.656      0.000      -1.905      -1.781
         online_reservation        1.1193      0.034     32.480      0.000       1.052       1.187
         selected_meal_plan       -0.0746      0.040     -1.844      0.065      -0.154       0.005
         lead_time_buckets         1.7089      0.029     58.399      0.000       1.652       1.766
         avg_price_per_room_buckets   0.7212      0.029     24.606      0.000       0.664       0.779
         length_of_stay_buckets   -0.0898      0.029     -3.070      0.002      -0.147      -0.032
         ==========================================================================================
```

Figure 59: Regression 2

Figure 59 represents a second attempt at an initial logistic regression model, however we can see that the results still failed to converge. When choosing between two highly collinear variables, the variable retained was the variable that showed greater influence over cancelation rate in the bivariate visualizations. While this regression result was still not successful, we can attempt to refine our variables further via other means.

```
In [98]: hotel_data_for_corr = hotel_data[['booking_status', 'required_car_parking_space','arrival_year', 'arrival_month', 'arrival_
                                 'count_of_guests', 'count_of_previous_bookings', 'prior_cancellation', 'special_request_flag',
                                 'online_reservation', 'selected_meal_plan', 'lead_time_buckets', 'avg_price_per_room_buckets',
                                 'length_of_stay_buckets']]
         corr_matrix = hotel_data_for_corr.corr()
         plt.figure(figsize=(16, 12))
         sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f")
         plt.show()
```

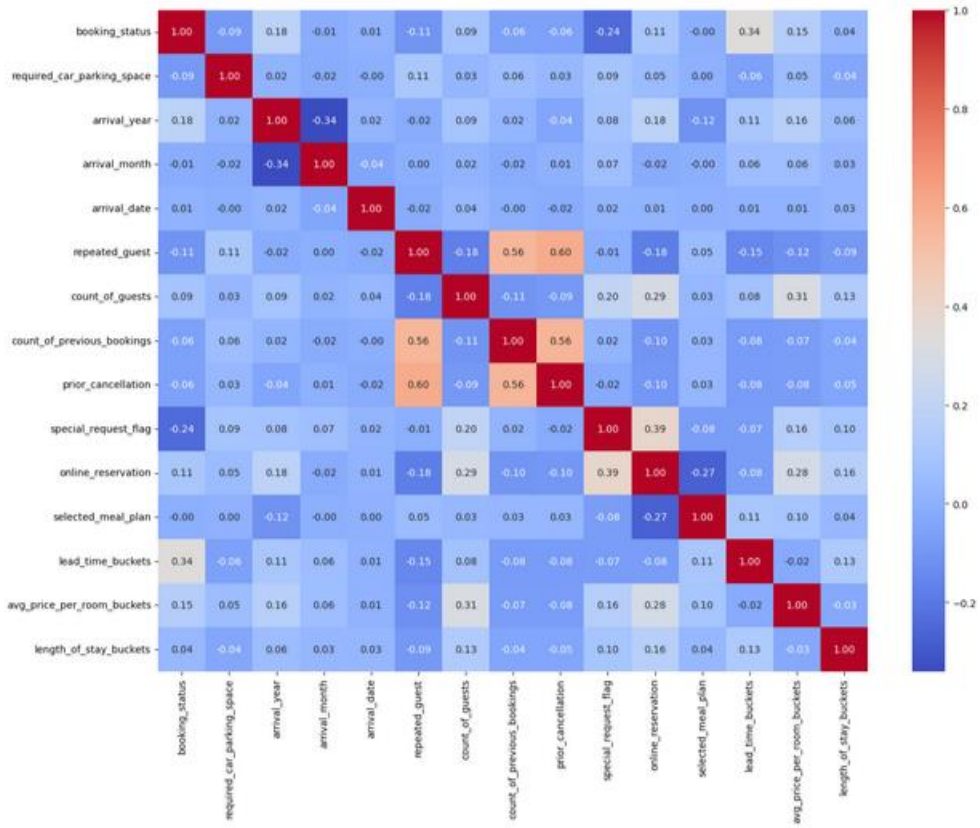

Figure 60: Correlation Matrix 2

We can begin the process of refining our model by creating another correlation heatmap. While there are values with relatively high correlation coefficients, no elements of multicollinearity jump out from the above matrix. However, we can use VIF to assess for any multicollinearity that may remain.

```
In [99]: #Feature selection via VIF
         hotel_data_vif = hotel_data_for_corr.drop(['booking_status'], axis=1)
         print(hotel_data_vif.columns)

         vif_data = pd.DataFrame()
         vif_data["feature"] = hotel_data_vif.columns

         vif_data["VIF"] = [variance_inflation_factor(hotel_data_vif.values, i)
         for i in range(len(hotel_data_vif.columns))]
         print(vif_data)

         Index(['required_car_parking_space', 'arrival_year', 'arrival_month',
                'arrival_date', 'repeated_guest', 'count_of_guests',
                'count_of_previous_bookings', 'prior_cancellation',
                'special_request_flag', 'online_reservation', 'selected_meal_plan',
                'lead_time_buckets', 'avg_price_per_room_buckets',
                'length_of_stay_buckets'],
               dtype='object')
                            feature        VIF
         0   required_car_parking_space   1.063676
         1                 arrival_year  27.468311
         2                arrival_month   6.989041
         3                 arrival_date   4.203395
         4               repeated_guest   1.914826
         5              count_of_guests  12.180620
         6   count_of_previous_bookings   1.670996
         7            prior_cancellation   1.797029
         8          special_request_flag   2.249881
         9            online_reservation   4.188630
         10           selected_meal_plan   8.102367
         11            lead_time_buckets   2.143051
         12  avg_price_per_room_buckets   2.222835
         13      length_of_stay_buckets   1.555042
```

Figure 61: VIF Feature Selection

VIF, or Variance Inflation Factor, can also be used to assess multicollinearity. According to Singh (2024), “VIF > 5: High multicollinearity is present. The predictor’s standard error may be noticeably inflated, which can make its coefficient less reliable. Consider taking steps to reduce multicollinearity, like removing or combining correlated predictors.” As such, any variable with a VIF above 5 will be removed, and multicollinearity will again be assessed. In this instance, this means that the Arrival Year, Arrival Month, Count of Guests, and Selected Meal Plan will be removed.

```
In [100]: #Remove VIF over 5
hotel_data_vif_2 = hotel_data_vif.drop(['arrival_year', 'arrival_month', 'count_of_guests', 'selected_meal_plan'], axi
hotel_data_vif_2.head()
vif_data = pd.DataFrame()
vif_data["feature"] = hotel_data_vif_2.columns

vif_data["VIF"] = [variance_inflation_factor(hotel_data_vif_2.values, i)
for i in range(len(hotel_data_vif_2.columns))]
print(vif_data)

                     feature       VIF
0  required_car_parking_space  1.062105
1                arrival_date  2.605990
2              repeated_guest  1.841136
3  count_of_previous_bookings  1.668989
4          prior_cancellation  1.795197
5        special_request_flag  2.177167
6          online_reservation  3.257122
7           lead_time_buckets  1.763778
8  avg_price_per_room_buckets  1.926911
9      length_of_stay_buckets  1.512352
```

Figure 62: VIF Feature Selection 2

After we remove the variable with VIFs above 5, we take a look at the VIF values again and note that the new highest VIF is approximately 3.26. It would appear that we have sufficiently addressed multicollinearity, and we can attempt to run another initial regression model.

```
In [101]: #Remove 'arrival_year', 'arrival_month', 'count_of_guests' 'selected_meal_plan' and try Logit model again
independent_variables = ['required_car_parking_space', 'arrival_date', 'repeated_guest',
                         'count_of_previous_bookings', 'prior_cancellation', 'special_request_flag',
                         'online_reservation', 'lead_time_buckets', 'avg_price_per_room_buckets',
                         'length_of_stay_buckets']
X = hotel_data[independent_variables]
y = hotel_data['booking_status']

X = sm.add_constant(X)
model2 = sm.Logit(y, X)
result2 = model2.fit()
print(result2.summary())

Warning: Maximum number of iterations has been exceeded.
         Current function value: 0.494648
         Iterations: 35
                           Logit Regression Results
==============================================================================
Dep. Variable:         booking_status   No. Observations:                36275
Model:                          Logit   Df Residuals:                    36264
Method:                           MLE   Df Model:                           10
Date:                Wed, 08 Jan 2025   Pseudo R-squ.:                  0.2179
Time:                        12:00:26   Log-Likelihood:                -17943.
converged:                       False   LL-Null:                       -22944.
Covariance Type:            nonrobust   LLR p-value:                     0.000
==============================================================================================
                                 coef    std err          z      P>|z|      [0.025      0.975]
----------------------------------------------------------------------------------------------
const                         -2.2242      0.040    -55.595      0.000      -2.303      -2.146
required_car_parking_space    -1.4558      0.111    -13.132      0.000      -1.673      -1.239
arrival_date                   0.0030      0.001      2.041      0.041       0.000       0.006
repeated_guest               -18.8129   1750.050     -0.011      0.991   -3448.848    3411.222
count_of_previous_bookings    -0.0193      0.031     -0.615      0.539      -0.081       0.042
prior_cancellation            17.7919   1750.050      0.010      0.992   -3412.243    3447.827
special_request_flag          -1.7888      0.031    -58.123      0.000      -1.849      -1.728
online_reservation             1.2819      0.032     39.724      0.000       1.219       1.345
lead_time_buckets              1.7756      0.029     62.148      0.000       1.720       1.832
avg_price_per_room_buckets     0.8387      0.027     30.612      0.000       0.785       0.892
length_of_stay_buckets        -0.0476      0.029     -1.659      0.097      -0.104       0.009
==============================================================================================
```

Figure 63: Regression after VIF Feature Removal

Figure 63 demonstrates the output of the logistic regression model after VIF feature removal has been performed. Unfortunately, the results still do not converge. However, the coefficient results appear to be a bit more promising, and we can identify variables with the largest p-values for removal via backward stepwise elimination.

According to Wormuth (2024), "In backward elimination, the model starts with all possible predictors and successively removes non-significant predictors until reaching the stopping criteria." In this instance, the stopping criteria will be until all independent variables have p-values less than 0.05. At each step, the least significant variable will be dropped from the model. The variable dropping process will continue until all remaining independent variables are significant at a 95% confidence level.

```
In [102]: #Feature selection via backward stepwise elimination (until all remaining are significant)
          ##Remove least significant variable and regress again (prior_cancellation)
          independent_variables = ['required_car_parking_space', 'arrival_date', 'repeated_guest',
                                   'count_of_previous_bookings', 'special_request_flag',
                                   'online_reservation', 'lead_time_buckets', 'avg_price_per_room_buckets',
                                   'length_of_stay_buckets']
          X = hotel_data[independent_variables]
          y = hotel_data['booking_status']

          X = sm.add_constant(X)
          model3 = sm.Logit(y, X)
          result3 = model3.fit()
          print(result3.summary())

          Optimization terminated successfully.
                   Current function value: 0.494947
                   Iterations 8
                                 Logit Regression Results
          ==============================================================================
          Dep. Variable:         booking_status   No. Observations:                36275
          Model:                          Logit   Df Residuals:                    36265
          Method:                           MLE   Df Model:                            9
          Date:                Wed, 08 Jan 2025   Pseudo R-squ.:                  0.2175
          Time:                        12:00:32   Log-Likelihood:                -17954.
          converged:                       True   LL-Null:                       -22944.
          Covariance Type:            nonrobust   LLR p-value:                     0.000
          ================================================================================================
                                             coef    std err          z      P>|z|      [0.025      0.975]
          ------------------------------------------------------------------------------------------------
          const                           -2.2252      0.040    -55.611      0.000      -2.304      -2.147
          required_car_parking_space      -1.4590      0.111    -13.167      0.000      -1.676      -1.242
          arrival_date                     0.0030      0.001      2.002      0.045    6.13e-05       0.006
          repeated_guest                  -1.9258      0.308     -6.248      0.000      -2.530      -1.322
          count_of_previous_bookings       0.0085      0.028      0.298      0.766      -0.047       0.064
          special_request_flag            -1.7905      0.031    -58.173      0.000      -1.851      -1.730
          online_reservation               1.2845      0.032     39.790      0.000       1.221       1.348
          lead_time_buckets                1.7776      0.029     62.190      0.000       1.722       1.834
          avg_price_per_room_buckets       0.8381      0.027     30.596      0.000       0.784       0.892
          length_of_stay_buckets          -0.0477      0.029     -1.663      0.096      -0.104       0.009
          ================================================================================================
```

Figure 64: Backward Stepwise Regression 1

After the least significant independent variable is dropped (Prior Cancellation), we can note that for the first time in this process, the results converge. Additionally, only two remaining

variables have p-values of greater than .05. We can perform another backward elimination, removing the number of previous bookings, and reassess the results.

```
In [103]: ##Remove least significant variable and regress again (count_of_previous_bookings)
          independent_variables = ['required_car_parking_space', 'arrival_date', 'repeated_guest',
                                   'special_request_flag', 'online_reservation', 'lead_time_buckets',
                                   'avg_price_per_room_buckets', 'length_of_stay_buckets']
          X = hotel_data[independent_variables]
          y = hotel_data['booking_status']

          X = sm.add_constant(X)
          model4 = sm.Logit(y, X)
          result4 = model4.fit()
          print(result4.summary())

          Optimization terminated successfully.
                   Current function value: 0.494948
                   Iterations 8
                                 Logit Regression Results
          ==============================================================================
          Dep. Variable:         booking_status   No. Observations:                36275
          Model:                          Logit   Df Residuals:                    36266
          Method:                           MLE   Df Model:                            8
          Date:                Wed, 08 Jan 2025   Pseudo R-squ.:                  0.2175
          Time:                        12:00:37   Log-Likelihood:                -17954.
          converged:                       True   LL-Null:                       -22944.
          Covariance Type:            nonrobust   LLR p-value:                     0.000
          ==============================================================================================
                                          coef    std err          z      P>|z|      [0.025      0.975]
          ----------------------------------------------------------------------------------------------
          const                        -2.2252      0.040    -55.612      0.000      -2.304      -2.147
          required_car_parking_space   -1.4589      0.111    -13.166      0.000      -1.676      -1.242
          arrival_date                  0.0030      0.001      2.002      0.045    6.27e-05       0.006
          repeated_guest               -1.8788      0.260     -7.238      0.000      -2.388      -1.370
          special_request_flag         -1.7904      0.031    -58.172      0.000      -1.851      -1.730
          online_reservation            1.2844      0.032     39.791      0.000       1.221       1.348
          lead_time_buckets             1.7775      0.029     62.190      0.000       1.722       1.834
          avg_price_per_room_buckets    0.8381      0.027     30.595      0.000       0.784       0.892
          length_of_stay_buckets       -0.0477      0.029     -1.663      0.096      -0.104       0.009
          ==============================================================================================
```

Figure 65: Backward Stepwise Regression 2

We perform another regression, and note that now we only have one variable with a p-value greater than .05. We can remove the Length of Stay buckets and regress again, hopefully achieving all significant results in our next attempt.

```
In [104]: ##Remove least significant variable and regress again (length_of_stay_buckets)
          independent_variables = ['required_car_parking_space', 'arrival_date', 'repeated_guest',
                                   'special_request_flag', 'online_reservation', 'lead_time_buckets',
                                   'avg_price_per_room_buckets']
          X = hotel_data[independent_variables]
          y = hotel_data['booking_status']

          X = sm.add_constant(X)
          model5 = sm.Logit(y, X)
          result5 = model5.fit()
          print(result5.summary())

          Optimization terminated successfully.
                   Current function value: 0.494986
                   Iterations 8
                                     Logit Regression Results
          ==============================================================================
          Dep. Variable:         booking_status   No. Observations:                36275
          Model:                          Logit   Df Residuals:                    36267
          Method:                           MLE   Df Model:                            7
          Date:                Wed, 08 Jan 2025   Pseudo R-squ.:                  0.2174
          Time:                        12:00:40   Log-Likelihood:                -17956.
          converged:                       True   LL-Null:                       -22944.
          Covariance Type:            nonrobust   LLR p-value:                     0.000
          ==========================================================================================
                                          coef    std err          z      P>|z|      [0.025      0.975]
          ------------------------------------------------------------------------------------------
          const                        -2.2305      0.040    -55.927      0.000      -2.309      -2.152
          required_car_parking_space   -1.4539      0.111    -13.127      0.000      -1.671      -1.237
          arrival_date                  0.0029      0.001      1.953      0.051   -1.02e-05       0.006
          repeated_guest               -1.8730      0.260     -7.217      0.000      -2.382      -1.364
          special_request_flag         -1.7929      0.031    -58.310      0.000      -1.853      -1.733
          online_reservation            1.2756      0.032     40.084      0.000       1.213       1.338
          lead_time_buckets             1.7713      0.028     62.536      0.000       1.716       1.827
          avg_price_per_room_buckets    0.8418      0.027     30.829      0.000       0.788       0.895
          ==========================================================================================
```

Figure 66: Backward Stepwise Regression 3

After the removal of what seemed like the only remaining insignificant independent variable, we see that Arrival Date is just outside the desired threshold. We will remove Arrival Date and regress again.

```
In [105]: ##Remove least significant variable and regress again (arrival_date)
          independent_variables = ['required_car_parking_space', 'repeated_guest',
                                   'special_request_flag', 'online_reservation', 'lead_time_buckets',
                                   'avg_price_per_room_buckets']
          X = hotel_data[independent_variables]
          y = hotel_data['booking_status']

          X = sm.add_constant(X)
          final_model = sm.Logit(y, X)
          final_result = final_model.fit()
          print(final_result.summary())

Optimization terminated successfully.
         Current function value: 0.495038
         Iterations 8
                           Logit Regression Results
==============================================================================
Dep. Variable:         booking_status   No. Observations:                36275
Model:                          Logit   Df Residuals:                    36268
Method:                           MLE   Df Model:                            6
Date:                Wed, 08 Jan 2025   Pseudo R-squ.:                  0.2173
Time:                        12:00:43   Log-Likelihood:                -17958.
converged:                       True   LL-Null:                       -22944.
Covariance Type:            nonrobust   LLR p-value:                     0.000
=================================================================================================
                                    coef    std err          z      P>|z|      [0.025      0.975]
-------------------------------------------------------------------------------------------------
const                            -2.1864      0.033    -66.706      0.000      -2.251      -2.122
required_car_parking_space       -1.4541      0.111    -13.131      0.000      -1.671      -1.237
repeated_guest                   -1.8772      0.260     -7.231      0.000      -2.386      -1.368
special_request_flag             -1.7919      0.031    -58.294      0.000      -1.852      -1.732
online_reservation                1.2764      0.032     40.114      0.000       1.214       1.339
lead_time_buckets                 1.7715      0.028     62.546      0.000       1.716       1.827
avg_price_per_room_buckets        0.8414      0.027     30.819      0.000       0.788       0.895
=================================================================================================
```

Figure 67: Backward Stepwise Regression 4

After removing the Arrival Date variable from the list of independent variables, we now have all of the independent variables as statistically significant. As a result, we can accept the current model as our final model and assess the model accuracy.

```
In [106]: #Confusion matrix
y_prob = final_result.predict(X)

##Convert probabilities to binary predictions based on a threshold
threshold = 0.5
y_pred = np.where(y_prob > threshold, 1, 0)

##Generate confusion matrix
cnf_matrix = confusion_matrix(y, y_pred)

##Visualize results using Heatmap (Code borrowed from https://www.datacamp.com/tutorial/understanding-logistic-regression-p
class_names=[0,1] #name of classes
fig, ax = plt.subplots()
tick_marks = np.arange(len(class_names))
plt.xticks(tick_marks, class_names)
plt.yticks(tick_marks, class_names)
##Create heatmap
sns.heatmap(pd.DataFrame(cnf_matrix), annot=True, cmap="YlGnBu", fmt='g')
ax.xaxis.set_label_position("top")
plt.tight_layout()
plt.title('Confusion matrix', y=1.1)
plt.ylabel('Actual label')
plt.xlabel('Predicted label')

Out[106]: Text(0.5, 427.9555555555555, 'Predicted label')
```

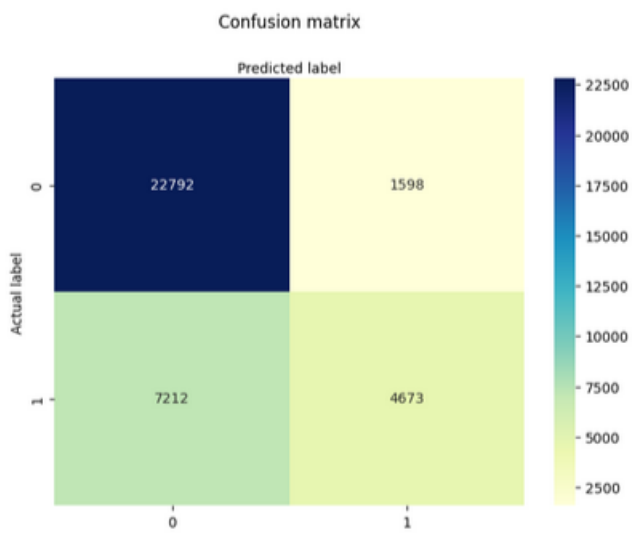

Figure 68: Confusion Matrix

Above, we generate a confusion matrix that will assess the number of True Positive, False Positive, True Negative, and False Negative results. By collecting these values, we can determine the accuracy score of the logistic regression model.

```
In [107]: #Calculate accuracy score
          TN = 22792
          FP = 1598
          FN = 7212
          TP = 4673

          accuracy_score = (TP+TN)/(TP + TN + FP + FN)
          print(accuracy_score)

          0.7571330117160578
```

Figure 69: Accuracy Score

In Figure 69 we are able to see that our model correctly predicts results approximately 75.7% of the time. This represents a solid result for the model, indicating that we have achieved a good starting point that can further be refined and assessed going forward.

Regarding the use of the backward stepwise elimination technique, one advantage of this process is that it reduces model complexity in a way that produces a simple and more easily interpreted model. Based on the results shown in Figure 67, we can easily interpret the variable coefficients in a way that will allow us to use the model in the Data Summary and Implications section.

One disadvantage of the backward stepwise elimination technique is that it is relatively resource intensive. Hayes (2022) explains that "Statisticians note several drawbacks to the approach, including incorrect results, an inherent bias in the process itself, and the necessity for significant computing power to develop complex regression models through iteration." We can see the relatively intensive resource power needed above by virtue of having to manually reassess the logistic regression after removing the least significant variable at each iteration. While this was not tremendously impactful in this instance, it is relatively easy to see this process becoming significantly more laborious when a much higher number of initial variables are used. Both in terms of time needed and computing power needed, backward stepwise elimination can potentially be a much more arduous process than others.

E. Data Summary and Implications

The research question that formed the basis of this data analysis was as follows: what factors most significantly influence hotel reservation cancellations, and how accurately can logistic regression predict the likelihood of a reservation being canceled? In the final logistic regression model, it was determined that the following factors most significantly influenced hotel

reservation cancellations: Whether the guest required a parking space, whether the guest was a repeated guest, whether the guest made a special request, whether the reservation was made online, whether the reservation had a High lead time, and whether the reservation had a High average price per room. In addition, the accuracy score of approximately 75.7% indicated that the logistic regression model was able to predict the likelihood of a reservation being canceled approximately 75.7% of the time.

One limitation of this analysis is that of the 36,275 total observations in the data, 24,390 did not cancel their reservation, while 11,885 did. This imbalance in the target variable can create potential issues with the logistic regression model. GeeksforGeeks (2024) states that "In imbalanced datasets, where the classes of interest are not equally represented, traditional logistic regression models may exhibit bias towards the majority class, leading to suboptimal performance, especially for predicting rare events."

One way to evaluate whether the imbalance in the target variable has impacted the model is to use performance metrics other than accuracy score. For example, tests such as precision or F1-score can be used to further assess model performance. As such, while there are potential limitations in the analysis, these limitations can be mitigated or further explored via additional testing.

Overall, the formula for the logistic regression model is as follows:

log-odds(P(booking_status=1))= −2.1864 − 1.4541*required_car_parking_space − 1.8772*repeated_guest − 1.7919*special_request_flag + 1.2764*online_reservation + 1.7715*lead_time_buckets + 0.8414*avg_price_per_room_buckets.

This log-odds formula can be used to identify the likelihood that a reservation is canceled in the most-likely and least-likely scenarios. The scenario in which a reservation is least likely to be canceled (with a cancelation likelihood of approximately 6%) is one in which the guest picks parking, is a repeated guest, has a special request, does not book on line, has a short lead time, and has a low price room. The scenario in which a reservation is most likely to be canceled (with a cancelation likelihood of approximately 84%) is one in which the guest does not pick parking, is not a repeated guest, has no special requests, makes an online reservation, as a long lead time, and has a high price room.

Within the context of the research question, the recommended course of action would be to attempt to use this model on future data to assess how well it performs outside the context of the original data. Several relatively applicable variables have been identified as impactful, and the relatively decent accuracy score indicates that using the model as-is could potentially have benefits for a hotel in terms of predicting whether a hotel reservation will be canceled.

The first direction or approach for future study would be to assess the model using different accuracy metrics in order to address any potential class imbalances currently present. If the model does not perform sufficiently based on these accuracy metrics, it could be beneficial to see if the model performs any different based on a balanced random sample from the initial data. If 5,000 instances of a reservation being canceled and 5,000 instances of a reservation not being canceled were randomly chosen and added to a dataset of 10,000 observations, the logistic regression model could be assessed for its performance on a balanced dataset.

The second direction or approach would be to interact variables in an attempt to parse out any relevant information that was missed by the current logistic regression attempt. The current model is relatively simple, and while it appeared to be effective at predicting whether a hotel reservation was canceled, it could be possible that additional valuable insights remain hidden in the data. As a whole, attempting to fine tune or reevaluate the model would likely increase the value of the currently developed logistic regression model approach.

References

2.2.4.1 - Skewness & Central tendency: Stat 200. PennState: Statistics Online Courses. (n.d.). https://online.stat.psu.edu/stat200/lesson/2/2.2/2.2.4/2.2.4.1

Allison, P. (2024, March 12). *When can you safely ignore multicollinearity?*. Statistical Horizons. https://statisticalhorizons.com/multicollinearity/

Filho, M. (2023, July 1). *How to solve logistic regression not converging in Scikit-learn*. Forecastegy (Alt + H). https://forecastegy.com/posts/how-to-solve-logistic-regression-not-converging-in-scikit-learn/#reasons-for-non-convergence-in-logistic-regression

GeeksforGeeks. (2024a, March 11). *Weighted Logistic Regression for Imbalanced Dataset*. https://www.geeksforgeeks.org/weighted-logistic-regression-for-imbalanced-dataset/

GeeksforGeeks. (2024b, April 10). *Outlier detection in logistic regression*. https://www.geeksforgeeks.org/outlier-detection-in-logistic-regression/

Hayes, A. (2022). *Stepwise regression: Definition, uses, example, and limitations*. Investopedia. https://www.investopedia.com/terms/s/stepwise-regression.asp

Littler, S. (2021, April 13). *Analysing categorical data using logistic regression models*. Select Statistical Consultants. https://select-statistics.co.uk/blog/analysing-categorical-data-using-logistic-regression-models/

Navlani, A. (2024, August 11). *Python logistic regression tutorial with Sklearn & Scikit*. DataCamp. https://www.datacamp.com/tutorial/understanding-logistic-regression-python

Raza, A. (2023, January 4). *Hotel Reservations dataset*. Kaggle. https://www.kaggle.com/datasets/ahsan81/hotel-reservations-classification-dataset/data

Schwartz, Z. (2021, January 1). *Consumers vs. revenue managers? the case of cancellations and no shows*. Boston Hospitality Review Consumers vs Revenue Managers The Case of Cancellations and No Shows Comments. https://www.bu.edu/bhr/2021/06/29/consumers-vs-revenue-managers-the-case-of-cancelations-and-no-shows/

Singh, V. (2024, November 18). *Variance inflation factor: How to detect multicollinearity*. DataCamp. https://www.datacamp.com/tutorial/variance-inflation-factor

Skift, O. C. +. (2024, August 28). *Summer Travel's Hidden Hurdle: The impacts of cancellations and no-shows*. https://skift.com/2024/07/23/summer-travels-hidden-hurdle-the-impacts-of-cancellations-and-no-shows/

Stojiljković, M. (2023, June 26). *Logistic regression in python*. Real Python. https://realpython.com/logistic-regression-python/#logistic-regression-python-packages

What is a data dictionary? A comprehensive guide. Splunk. (2024). https://www.splunk.com/en_us/blog/learn/data-dictionary.html

Wormuth, B. (2024, December 23). *Stepwise regression: What is it and should you use it?*. Statistics Solutions. https://www.statisticssolutions.com/stepwise-regression-what-is-it-and-should-you-use-it/